

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN 2

KHAI THÁC TẬP PHỔ BIẾN

PHÂN TÍCH THUẬT TOÁN FP-GROWTH VÀ KHAI THÁC TẬP MỤC PHỔ BIẾN

Môn học: Khai thác dữ liệu và ứng dụng
Mã học phần: CSC14004
Học kỳ: Học kỳ 2 – 2025–2026
Giảng viên hướng dẫn: Th.S Nguyễn Thị Thu Hằng
Th.S Nguyễn Ngọc Đức
Th.S Lê Nhật Nam

Nhóm sinh viên thực hiện:

Nguyễn Hữu Khánh Hưng	23120271
Phạm Quốc Khánh	23120283
Y Nguyễn Mlô	23120299
Vũ Trần Phúc	23120333
Trần Nguyễn Minh Quân	23120342

TP. Hồ Chí Minh, tháng 6 năm 2025

Lời cảm ơn

Nhóm chúng em xin gửi lời cảm ơn chân thành đến đội ngũ giảng viên phụ trách môn *Khai thác dữ liệu và ứng dụng*, đặc biệt là các thầy/cô đã tận tình hỗ trợ và định hướng trong suốt quá trình thực hiện báo cáo. Những bài giảng, góp ý chuyên môn và góc nhìn thực tiễn của giảng viên giúp nhóm hiểu rõ hơn ý nghĩa của từng kỹ thuật trong bài toán khai thác frequent itemset.

Báo cáo này được thực hiện với mục tiêu trình bày có hệ thống nền tảng lý thuyết của bài toán khai thác frequent itemset, nguyên lý hoạt động của thuật toán FP-Growth, cùng các ví dụ minh họa chi tiết nhằm làm rõ quá trình xây dựng FP-tree và khai thác mẫu phổ biến. Nhóm đã cố gắng bảo đảm tính chính xác của ký hiệu toán học, tính logic trong diễn giải và tính nhất quán trong trình bày.

Mặc dù đã nỗ lực hoàn thiện báo cáo một cách cẩn chu nhất, nhóm vẫn khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được các ý kiến góp ý từ quý thầy/cô để tiếp tục hoàn thiện kiến thức và kỹ năng học thuật của mình.

TP. Hồ Chí Minh, tháng 4 năm 2025

Nhóm sinh viên thực hiện

Contents

Lời cảm ơn	i
Bảng thuật ngữ	vi
1 Nền tảng lý thuyết	1
1.1 Bài toán khai thác frequent itemset	1
1.1.1 Cơ sở dữ liệu giao dịch	1
1.1.2 support	1
1.1.3 Frequent itemset	2
1.1.4 Tập đóng và tập tối đại	2
1.1.5 Tính chất đơn điệu giảm của support	3
1.2 Phân tích thuật toán FP-Growth	4
1.2.1 Ý tưởng cốt lõi	4
1.2.2 Cấu trúc dữ liệu chính: FP-tree	4
1.2.3 Giả mã	5
1.2.4 Phân tích độ phức tạp	7
1.2.5 So sánh lịch sử	8
2 Ví dụ minh họa tay	9
2.1 Ví dụ 1: Trường hợp cơ sở	9
2.1.1 Cơ sở dữ liệu gốc	9
2.1.2 Bước 1: Quét lần 1 — Tìm items phổ biến cấp một	9
2.1.3 Bước 2: Quét lần 2 — Xây dựng FP-tree	9
2.1.4 Bước 3: Khai thác FP-tree — Xây dựng conditional pattern-base	11
2.1.5 Bước 4: Tổng hợp kết quả	13
2.1.6 Bước 5: Kiểm tra chéo	13
2.2 Ví dụ 2: Tình huống đặc biệt — FP-tree đường đơn	14
2.2.1 Mô tả tình huống đặc biệt	14
2.2.2 Cơ sở dữ liệu gốc	14

2.2.3	Bước 1: Quét lần 1 — Tìm items phổ biến cấp một	15
2.2.4	Bước 2: Quét lần 2 — Xây dựng FP-tree	15
2.2.5	Bước 3: Khai thác — Áp dụng tối ưu hóa đường đơn	16
2.2.6	Bước 4: Phân tích tình huống đặc biệt	17
3	Cài đặt thuật toán	20
3.1	Tổng quan kiến trúc dự án	20
3.1.1	Môi trường và công cụ	20
3.1.2	Cấu trúc thư items	20
3.2	Cài đặt cơ bản	22
3.2.1	Cấu trúc dữ liệu FP-tree	22
3.2.2	Xây dựng FP-tree	22
3.2.3	Khai thác đệ quy	23
3.2.4	Đọc và ghi dữ liệu theo định dạng SPMF	24
3.3	Tối ưu hóa	24
3.3.1	Cắt tỉa đường đơn	24
3.3.2	Lọc bằng mảng bit	25
3.3.3	Tái sử dụng bộ đệm	25
3.3.4	Ổn định kiểu và nội tuyến hóa	25
3.4	Kiểm thử tự động	25
3.4.1	Quy trình kiểm thử	26
3.5	So sánh phiên bản cơ bản và phiên bản tối ưu	26
3.6	Giao diện dòng lệnh	27
3.7	Tóm tắt	28
4	Thực nghiệm và đánh giá	29
4.1	Tập dữ liệu chuẩn	29
4.1.1	Connect-4	29
4.1.2	T10I4D100K	30
4.1.3	T20I6D100K	30
4.1.4	Accidents	30
4.1.5	Retail	30

4.2	Kiểm tra tính đúng đắn	31
4.2.1	Kiểm thử đơn vị	31
4.2.2	Kiểm chứng trên tập chuẩn	31
4.3	Thời gian chạy theo minsup	32
4.3.1	Connect-4	33
4.3.2	T10I4D100K	34
4.3.3	T20I6D100K	35
4.3.4	Accidents	36
4.3.5	Retail	38
4.4	Số lượng frequent itemset theo minsup	39
4.4.1	Connect-4	39
4.4.2	T10I4D100K	40
4.4.3	T20I6D100K	41
4.4.4	Accidents	41
4.4.5	Retail	42
4.4.6	So sánh giữa dữ liệu dày đặc và dữ liệu thưa	43
4.5	Sử dụng bộ nhớ	43
4.6	Khả năng mở rộng	44
4.7	Ảnh hưởng của độ dài giao dịch	45
4.8	Phân tích tổng hợp kết quả	46
4.8.1	Giải thích kết quả dựa trên lý thuyết	46
4.8.2	Điểm mạnh và điểm yếu so với SPMF	46
4.8.3	Hướng tối ưu hóa tiếp theo	47
5	Ứng dụng — Market Basket Analysis	48
5.1	Giới thiệu bài toán	48
5.2	Tập dữ liệu và tiền xử lý	48
5.2.1	Mô tả tập dữ liệu	48
5.2.2	Chiến lược gom giao dịch	49
5.2.3	Mã hóa item	49
5.2.4	Phân phối kích thước giỏ hàng	49
5.2.5	Phân tích thống kê theo thời gian	50

5.3	Lựa chọn ngưỡng tham số	50
5.4	Khai thác frequent itemset	51
5.5	Sinh association rule	51
5.6	10 rule theo lift	52
5.6.1	Phân tích ý nghĩa kinh doanh	52
5.7	Trực quan hóa	54
5.8	Hạn chế và hướng mở rộng	57
5.8.1	Hạn chế của mô hình	57
5.8.2	Hướng mở rộng	58
Tài liệu tham khảo		59

Bảng thuật ngữ

Bảng thuật ngữ dưới đây tổng hợp các khái niệm chính được sử dụng trong báo cáo. Bảng gồm thuật ngữ gốc, cách diễn đạt tiếng Việt tương ứng và phần giải thích ngắn gọn, nhằm giúp người đọc tra cứu thống nhất trong suốt tài liệu.

Bảng 1: Bảng thuật ngữ chính

Thuật ngữ gốc	Thuật ngữ tiếng Việt	Giải thích
Item	Mục	Một phần tử đơn lẻ thuộc tập mục $\mathcal{I}$.
Itemset	Tập mục	Một tập con không rỗng của tập mục $\mathcal{I}$.
Transaction	Giao dịch	Một bản ghi chứa các mục xuất hiện đồng thời.
Transaction Database	Cơ sở dữ liệu giao dịch	Tập hợp toàn bộ các giao dịch được khai thác.
Support	Độ hỗ trợ	Số giao dịch chứa một tập mục cho trước.
Relative Support	Độ hỗ trợ tương đối	Tỷ lệ giao dịch chứa tập mục trên tổng số giao dịch.
Minimum Support Threshold	Ngưỡng hỗ trợ tối thiểu	Ngưỡng dùng để xác định một tập mục có phổ biến hay không.
Frequent Itemset	Tập mục phổ biến	Tập mục có độ hỗ trợ không nhỏ hơn ngưỡng tối thiểu.
Closed Itemset	Tập mục đóng	Tập phổ biến không có tập cha thực sự nào cùng độ hỗ trợ.
Maximal Itemset	Tập mục tối đại	Tập phổ biến không có tập cha thực sự nào vẫn còn phổ biến.

Bảng 1 được tiếp tục

Thuật ngữ gốc	Thuật ngữ tiếng Việt	Giải thích
Association Rule	Luật kết hợp	Quan hệ dạng $X \Rightarrow Y$ mô tả việc tập mục X thường kéo theo tập mục Y .
Antecedent	Tiền đề	Vế trái X của một luật kết hợp $X \Rightarrow Y$.
Consequent	Hệ quả	Vế phải Y của một luật kết hợp $X \Rightarrow Y$.
Confidence	Độ tin cậy	Xác suất giao dịch chứa Y khi đã chứa X trong luật $X \Rightarrow Y$.
Lift	Chỉ số nâng	Chỉ số đo mức độ phụ thuộc giữa tiền đề và hệ quả so với trường hợp độc lập.
Market Basket Analysis	Phân tích giỏ hàng	Ứng dụng khai thác luật kết hợp để phát hiện các sản phẩm thường được mua cùng nhau.
Downward Closure Property	Tính chất đóng hướng xuống	Nếu một tập mục phổ biến thì mọi tập con của nó cũng phổ biến.
Anti-monotone Property	Tính đơn điệu giảm	Độ hỗ trợ không tăng khi mở rộng tập mục.
FP-tree	Cây FP	Cấu trúc cây tiền tố nén dùng để lưu thông tin tập mục phổ biến.
Header Table	Bảng tiêu đề	Bảng lưu các mục phổ biến cùng liên kết đến các nút tương ứng trong FP-tree.
Node-link	Liên kết nút	Con trỏ nối các nút mang cùng tên mục trong FP-tree.

Bảng 1 được tiếp tục

Thuật ngữ gốc	Thuật ngữ tiếng Việt	Giải thích
Conditional Pattern-base	Cơ sở mẫu có điều kiện	Tập các đường tiền tố đã điều chỉnh bộ đếm để khai thác theo từng mục.
Conditional FP-tree	Cây FP có điều kiện	Cây FP được xây dựng từ cơ sở mẫu có điều kiện.
Pattern Growth	Tăng trưởng mẫu	Phương pháp mở rộng mẫu bằng cách nối thêm hậu tố.
Single Prefix Path	Đường tiền tố đơn	Trường hợp FP-tree chỉ có một đường duy nhất từ gốc đến lá.
Candidate Generation	Sinh tập ứng viên	Quá trình tạo các tập mục ứng viên để kiểm tra độ hỗ trợ.
Divide and Conquer	Chia để trị	Chiến lược tách bài toán lớn thành các bài toán con nhỏ hơn.
Prefix Tree	Cây tiền tố	Cấu trúc cây khai thác sự chia sẻ tiền tố chung giữa các giao dịch.
Pruning	Cắt tỉa	Loại bỏ các nhánh tìm kiếm không cần thiết.

1. Nền tảng lý thuyết

1.1 Bài toán khai thác frequent itemset

1.1.1 Cơ sở dữ liệu giao dịch

Định nghĩa 1.1.1 (Cơ sở dữ liệu giao dịch). Cho $\mathcal{I} = \{i_1, i_2, \dots, i_m\}$ là tập hữu hạn tất cả các items. Một **cơ sở dữ liệu giao dịch** là một tập hữu hạn:

$$\mathcal{D} = \{T_1, T_2, \dots, T_n\}$$

trong đó $\mathcal{I}$ là tập tất cả các items, $m = |\mathcal{I}|$ là số items phân biệt, $n = |\mathcal{D}|$ là số giao dịch, và mỗi giao dịch T_k ($k \in \{1, 2, \dots, n\}$) là một tập con của $\mathcal{I}$, tức là $T_k \subseteq \mathcal{I}$. Mỗi giao dịch T_k được gán một định danh duy nhất gọi là TID.

Một **itemset** X là một tập con không rỗng của $\mathcal{I}$, tức $X \subseteq \mathcal{I}$ và $X \neq \emptyset$. Nếu $|X| = k$ thì X được gọi là k -itemset. Ta nói giao dịch T_k **chứa** itemset X khi và chỉ khi $X \subseteq T_k$.

1.1.2 support

Định nghĩa 1.1.2 (absolute support). Cho itemset $X \subseteq \mathcal{I}$ và cơ sở dữ liệu giao dịch $\mathcal{D}$. **absolute support** của X trong $\mathcal{D}$ là số giao dịch trong $\mathcal{D}$ chứa X :

$$\text{sup}(X) = |\{T \in \mathcal{D} \mid X \subseteq T\}|$$

trong đó $\text{sup}(X)$ là absolute support của itemset X , còn ký hiệu $|\cdot|$ biểu thị số phần tử của một tập hữu hạn.

Định nghĩa 1.1.3 (relative support). **relative support** của itemset X trong $\mathcal{D}$ là tỷ lệ giữa số giao dịch chứa X và tổng số giao dịch:

$$\text{rsup}(X) = \frac{\text{sup}(X)}{|\mathcal{D}|} = \frac{|\{T \in \mathcal{D} \mid X \subseteq T\}|}{n}$$

trong đó $\text{rsup}(X)$ là relative support của X , $\text{sup}(X)$ là absolute support, và $n = |\mathcal{D}|$ là tổng số giao dịch.

Sự phân biệt giữa hai dạng này rất quan trọng trong thực hành: absolute support cho biết số lượng giao dịch cụ thể, trong khi relative support cho phép so sánh giữa các cơ sở dữ liệu có kích thước khác nhau.

1.1.3 Frequent itemset

Định nghĩa 1.1.4 (Frequent itemset). Cho minimum support threshold ξ . Một itemset $X \subseteq \mathcal{I}$ được gọi là **phổ biến** trong $\mathcal{D}$ nếu và chỉ nếu:

$$\text{sup}(X) \geq \xi$$

trong đó ξ là minimum support threshold do người phân tích lựa chọn.

Tập tất cả các frequent itemset được ký hiệu là:

$$\mathcal{F} = \{X \subseteq \mathcal{I} \mid \text{sup}(X) \geq \xi\}$$

trong đó $\mathcal{F}$ là họ tất cả các frequent itemset trong cơ sở dữ liệu giao dịch $\mathcal{D}$.

Bài toán khai thác frequent itemset: Cho cơ sở dữ liệu giao dịch $\mathcal{D}$ và ngưỡng ξ , tìm tập đầy đủ $\mathcal{F}$ chứa tất cả frequent itemset cùng với support tương ứng.

1.1.4 Tập đóng và tập tối đại

Định nghĩa 1.1.5 (Frequent itemset đóng). Một frequent itemset X được gọi là **đóng** nếu không tồn tại frequent itemset Y nào sao cho:

$$X \subset Y \quad \text{và} \quad \text{sup}(X) = \text{sup}(Y)$$

trong đó Y là một tập cha thực sự của X . Nói cách khác, X là đóng nếu mọi tập cha thực sự của X đều có support nhỏ hơn nghiêm ngặt so với X . Tập tất cả frequent itemset đóng được ký hiệu là $\mathcal{C}$.

Định nghĩa 1.1.6 (Frequent itemset tối đại). Một frequent itemset X được gọi là **tối đại** nếu không tồn tại frequent itemset Y nào sao cho:

$$X \subset Y$$

trong đó Y là một tập cha thực sự của X . Nói cách khác, X là tối đại nếu mọi tập cha thực sự của X đều không phổ biến. Tập tất cả frequent itemset tối đại được ký hiệu là $\mathcal{M}$.

Định lý 1.1.1 (Quan hệ bao hàm). Với $\mathcal{F}$, $\mathcal{C}$, $\mathcal{M}$ lần lượt là tập các frequent itemset, frequent itemset đóng và frequent itemset tối đại, ta có:

$$\mathcal{M} \subseteq \mathcal{C} \subseteq \mathcal{F}$$

trong đó $\mathcal{M}$ là tập các frequent itemset tối đại, $\mathcal{C}$ là tập các frequent itemset đóng, và $\mathcal{F}$ là tập tất cả các frequent itemset.

Proof. **Phần 1:** $\mathcal{M} \subseteq \mathcal{C}$. Cho $X \in \mathcal{M}$. Giả sử phản chứng rằng $X \notin \mathcal{C}$. Khi đó tồn tại Y sao cho $X \subset Y$ và $\text{sup}(X) = \text{sup}(Y)$. Vì $\text{sup}(Y) = \text{sup}(X) \geq \xi$ nên Y là phổ biến, đồng thời $X \subset Y$ với Y phổ biến, mâu thuẫn với giả thiết X tối đại. Vậy $X \in \mathcal{C}$.

Phần 2: $\mathcal{C} \subseteq \mathcal{F}$. Theo định nghĩa, frequent itemset đóng trước hết phải là frequent itemset, kèm thêm điều kiện đóng. Do đó hiển nhiên $\mathcal{C} \subseteq \mathcal{F}$. $\square$

Ý nghĩa thực tiễn: Từ $\mathcal{M}$, ta có thể khôi phục tất cả frequent itemset nhưng không khôi phục được support chính xác. Từ $\mathcal{C}$, ta có thể khôi phục cả frequent itemset lẫn support chính xác thông qua phép lấy bao đóng.

1.1.5 Tính chất đơn điệu giảm của support

Tính chất 1.1.1 (Anti-monotone property). Nếu $X \subseteq Y$ thì $\text{sup}(X) \geq \text{sup}(Y)$.

Tương đương: Nếu một itemset X là phổ biến, thì mọi tập con $X' \subseteq X$ cũng phổ biến. Ngược lại, nếu X không phổ biến thì mọi tập cha $Y \supseteq X$ cũng không phổ biến.

Proof. Cho $X \subseteq Y$. Xét tập $S_Y = \{T \in \mathcal{D} \mid Y \subseteq T\}$. Với mỗi $T \in S_Y$, vì $Y \subseteq T$ và $X \subseteq Y$, theo tính bắc cầu ta suy ra $X \subseteq T$. Do đó $T \in S_X = \{T \in \mathcal{D} \mid X \subseteq T\}$.

Vậy $S_Y \subseteq S_X$, suy ra $|S_Y| \leq |S_X|$, tức là:

$$\text{sup}(Y) \leq \text{sup}(X)$$

trong đó $X \subseteq Y$, nên khi mở rộng itemset từ X sang Y , support không thể tăng.

Consequent trực tiếp: Nếu $\text{sup}(X) \geq \xi$ và $X' \subseteq X$, thì $\text{sup}(X') \geq \text{sup}(X) \geq \xi$, nên X' cũng phổ biến. Nếu $\text{sup}(X) < \xi$ và $Y \supseteq X$, thì $\text{sup}(Y) \leq \text{sup}(X) < \xi$, nên Y cũng không phổ biến. $\square$

Tính chất này là nền tảng cốt lõi không gian tìm kiếm trong tất cả các thuật toán khai thác frequent itemset. Trong FP-Growth, tính chất này được khai thác gián tiếp khi chỉ giữ lại các items phổ biến trong quá trình xây dựng FP-tree.

1.2 Phân tích thuật toán FP-Growth

1.2.1 Ý tưởng cốt lõi

Thuật toán FP-Growth được đề xuất bởi Han, Pei và Yin vào năm 2000, với phiên bản đầy đủ công bố năm 2004. Thuật toán giải quyết bài toán khai thác frequent itemset theo triết lý hoàn toàn khác so với hướng tiếp cận sinh-và-kiểm-tra truyền thống: khai thác frequent itemset mà không cần sinh candidate itemset.

Ý tưởng cốt lõi dựa trên ba kỹ thuật tích hợp:

Kỹ thuật 1 — Nén cơ sở dữ liệu: Toàn bộ cơ sở dữ liệu giao dịch được nén vào một cấu trúc cây tiền tố mở rộng gọi là FP-tree. Cấu trúc này chỉ cần đúng hai lần quét cơ sở dữ liệu: lần đầu để tìm các items phổ biến cấp một, lần hai để xây dựng cây. Sau đó, toàn bộ quá trình khai thác chỉ làm việc trên FP-tree mà không cần quay lại cơ sở dữ liệu gốc.

Kỹ thuật 2 — Pattern growth: Thay vì sinh candidate itemset rồi kiểm tra, FP-Growth sử dụng phương pháp tăng trưởng mảnh mẫu: bắt đầu từ mỗi items phổ biến cấp một, xây dựng conditional pattern-base, rồi đệ quy xây FP-tree con và khai thác tiếp.

Kỹ thuật 3 — Chia để trị: Bài toán khai thác được phân rã thành các bài toán con nhỏ hơn, mỗi bài toán con chỉ xử lý một cơ sở dữ liệu có điều kiện liên quan đến một items cụ thể, giảm mạnh không gian tìm kiếm.

Điểm khác biệt cốt lõi so với Apriori: Apriori phải quét cơ sở dữ liệu nhiều lần, mỗi mức k quét một lần, và sinh số lượng khổng lồ candidate itemset. Với 10^4 items phổ biến cấp một, Apriori phải sinh hơn 10^7 ứng viên cấp hai. Để tìm một frequent itemset kích thước 100, cần sinh tổng cộng $2^{100} - 2 \approx 10^{30}$ ứng viên. FP-Growth loại bỏ hoàn toàn bước sinh ứng viên, thay thế bằng phép đếm tích lũy và điều chỉnh đường tiền tố — các phép toán ít tốn kém hơn rất nhiều.

1.2.2 Cấu trúc dữ liệu chính: FP-tree

Định nghĩa 1.2.1 (FP-tree). *Một FP-tree là một cấu trúc cây bao gồm:*

1. *Một nút gốc được gán nhãn “null”, một tập các cây con tiền tố items là con của nút gốc, và một header table items phổ biến.*
2. *Mỗi nút trong cây con tiền tố items gồm ba trường:*

- *item-name: tên items mà nút đại diện.*

- **count**: số giao dịch được biểu diễn bởi phần đường dẫn đến nút này.
 - **node-link**: con trỏ đến nút tiếp theo trong FP-tree có cùng tên items, hoặc null nếu không có.
3. Mỗi items trong header table gồm hai trường: tên items và con trỏ đến nút đầu tiên trong FP-tree mang tên items đó.

FP-tree được xây dựng dựa trên các quan sát sau: chỉ các items phổ biến mới đóng vai trò trong khai thác, nên chỉ chúng được đưa vào cây; các items được sắp xếp theo thứ tự support giảm dần để tăng khả năng chia sẻ tiền tố; và nhiều giao dịch có thể dùng chung một đường dẫn nếu chúng chia sẻ cùng tiền tố.

Hai tính chất quan trọng của FP-tree là tính đầy đủ — mỗi giao dịch được ánh xạ thành đúng một đường dẫn, đảm bảo không mất thông tin — và tính nén — kích thước cây bị chặn trên bởi $\sum_{T \in \mathcal{D}} |\text{freq}(T)|$ nhưng trong thực tế thường nhỏ hơn nhiều bậc nhờ chia sẻ tiền tố.

1.2.3 Giải mã

Xây dựng FP-tree

Algorithm 1 Xây dựng FP-tree

Require: Cơ sở dữ liệu giao dịch $\mathcal{D}$, minimum support threshold ξ

Ensure: FP-tree $\mathcal{T}$

- 1: **Quét lần 1:** Duyệt $\mathcal{D}$, thu thập tập F các items phổ biến cấp một cùng support
 - 2: Sắp xếp F theo support giảm dần, gọi là FList
 - 3: Tạo nút gốc $\mathcal{T}$ với nhãn “null”
 - 4: **Quét lần 2: for each** giao dịch $T \in \mathcal{D}$ **do**
 - 5: Chọn và sắp xếp các items phổ biến trong T theo FList, gọi là $[p \mid P]$
 - 6: Gọi CHÈN_CÂY($[p \mid P], \mathcal{T}$)
 - 7: **end for**
 - 8: **return** $\mathcal{T}$
-

Algorithm 2 Chèn_Cây($[p \mid P], N$)

Require: Danh sách items $[p \mid P]$ với p là phần tử đầu và P là phần còn lại; nút N

- 1: **if** N có nút con C sao cho $C.\text{item-name} = p$ **then**
- 2: $C.\text{count} \leftarrow C.\text{count} + 1$ {Chia sẻ tiền tố, chỉ tăng bộ đếm}
- 3: **else**
- 4: Tạo nút mới C với $C.\text{item-name} = p$, $C.\text{count} = 1$
- 5: Liên kết C làm con của N ; cập nhật node-link trong header table
- 6: **end if**
- 7: **if** $P \neq \emptyset$ **then**
- 8: Gọi CHÈN_CÂY(P, C) {Đệ quy với phần còn lại}
- 9: **end if**

Thuật toán FP-Growth**Algorithm 3** FP-Growth

Require: FP-tree $\mathcal{T}$, itemset hậu tố α (ban đầu $\alpha = \emptyset$)

Ensure: Tập đầy đủ các frequent itemset

- 1: **if** $\mathcal{T}$ chứa một đường đơn P **then**
- 2: **for each** tổ hợp β của các nút trong P **do**
- 3: Sinh mẫu $\beta \cup \alpha$ với $\text{sup} = \min_{i \in \beta} \{i.\text{count}\}$
- 4: **end for**
- 5: **else**
- 6: **for each** items a_i trong header table của $\mathcal{T}$ (từ support thấp đến cao) **do**
- 7: Sinh mẫu $\beta = a_i \cup \alpha$ với $\text{sup}(\beta) = a_i.\text{support}$
- 8: Xây dựng conditional pattern-base B của a_i :
- 9: $B \leftarrow$ tập các đường tiền tố đã biến đổi của a_i
- 10: Xây dựng conditional FP-tree $\mathcal{T}_\beta$ từ B với ngưỡng ξ
- 11: **if** $\mathcal{T}_\beta \neq \emptyset$ **then**
- 12: Gọi FP-GROWTH($\mathcal{T}_\beta, \beta$)
- 13: **end if**
- 14: **end for**
- 15: **end if**

Ý nghĩa từng bước quan trọng:

Dòng 1–4: Khi FP-tree chỉ gồm một đường đơn, mọi tổ hợp con các nút trên đường đó đều là frequent itemset, với support bằng giá trị nhỏ nhất trong tổ hợp. Đây là tối ưu hóa quan trọng tránh đệ quy không cần thiết.

Dòng 6: Duyệt header table từ items có support thấp nhất đảm bảo tính chọn lọc cao, tạo cơ sở dữ liệu có điều kiện nhỏ nhất ngay từ đầu.

Dòng 8–9: Theo tính chất đường tiền tố, để tính tất cả frequent itemset chứa items a_i , ta chỉ cần thu thập các đường tiền tố đã biến đổi — đường từ gốc đến nút cha của mỗi nút a_i , với bộ đếm bằng bộ đếm tại nút a_i .

Dòng 10: Xây dựng FP-tree mới từ conditional pattern-base, loại bỏ các items dưới ngưỡng ξ . Cây con này thường nhỏ hơn rất nhiều so với cây ban đầu.

Dòng 11–12: Áp dụng hệ quả pattern growth: $\alpha \cup \beta$ phổ biến trong $\mathcal{D}$ khi và chỉ khi β phổ biến trong conditional pattern-base của α . Quá trình đệ quy biến bài toán tìm k -frequent itemset thành chuỗi các bài toán tìm 1-frequent itemset.

1.2.4 Phân tích độ phức tạp

Độ phức tạp thời gian

Trong trường hợp tốt nhất, khi dữ liệu rất thưa và hầu hết conditional FP-tree rỗng ngay ở mức đệ quy đầu tiên, độ phức tạp là $O(n \cdot \bar{w})$ với n là số giao dịch và $\bar{w}$ là chiều rộng trung bình. Chi phí chủ yếu nằm ở hai lần quét cơ sở dữ liệu để xây FP-tree.

Trong trường hợp trung bình, tổng chi phí tỷ lệ với tổng kích thước tất cả conditional FP-tree được sinh ra trong quá trình đệ quy. Giá trị này phụ thuộc mạnh vào đặc trưng dữ liệu, đặc biệt là mức độ chia sẻ tiền tố và số lượng items phổ biến.

Trong trường hợp xấu nhất, khi mọi items đều xuất hiện cùng nhau trong mọi giao dịch, số frequent itemset là $2^{|F|} - 1$ và thuật toán phải liệt kê toàn bộ, dẫn đến độ phức tạp $O(2^{|F|})$. Tuy nhiên, đây cũng là giới hạn dưới lý thuyết của bất kỳ thuật toán khai thác frequent itemset nào.

Độ phức tạp không gian

Kích thước FP-tree bị chặn trên bởi $\sum_{T \in \mathcal{D}} |\text{freq}(T)|$. Trong thực nghiệm, tỷ lệ nén có thể đạt hàng trăm lần. Không gian bổ sung cho quá trình đệ quy là $O(|F| \cdot S_{\max})$ trong trường hợp xấu nhất, với $S_{\max}$ là kích thước conditional FP-tree lớn nhất, nhưng thực tế các cây con ở mức sâu thường rất nhỏ.

Các yếu tố ảnh hưởng chính đến hiệu năng

Minimum support threshold là yếu tố ảnh hưởng mạnh nhất: khi ngưỡng giảm, số items phổ biến tăng, FP-tree lớn và sâu hơn, số conditional FP-tree nhiều hơn. Kích thước cơ sở dữ liệu ảnh hưởng tuyến tính đến chi phí hai lần quét ban đầu. Chiều rộng giao dịch trung bình quyết định chiều cao cây và chi phí đệ quy. Mật độ dữ liệu ảnh hưởng hai chiều: dữ liệu dày đặc cho phép nén tốt hơn nhưng cũng tạo nhiều frequent itemset hơn. Phân bố support đều giảm hiệu quả sắp xếp theo thứ tự giảm dần.

1.2.5 So sánh lịch sử

FP-Growth nằm ở vị trí bước ngoặt trong dòng thời gian phát triển thuật toán khai thác frequent itemset. Trước FP-Growth, các thuật toán chính bao gồm: AIS năm 1993 đặt nền móng bài toán; Apriori năm 1994 khai thác anti-monotone property để cắt tỉa; Partition năm 1995 giảm số lần quét cơ sở dữ liệu; Eclat năm 1997 sử dụng biểu diễn dọc với phép giao tập giao dịch; MaxMiner năm 1998 khai thác frequent itemset tối đại.

FP-Growth năm 2000 giải quyết hai hạn chế nghiêm trọng của Apriori: chi phí sinh ứng viên khổng lồ và quét cơ sở dữ liệu nhiều lần. So với Eclat, FP-Growth nén thông tin hiệu quả hơn nhờ chia sẻ tiền tố, đặc biệt khi cơ sở dữ liệu lớn và các tập giao dịch dài.

Sau FP-Growth, các thuật toán tiếp tục phát triển: TreeProjection năm 2001 sử dụng chiếu cây từ điển; Charm năm 2002 khai thác tập đóng với phép giao tập giao dịch vi phân; LCM năm 2004 đạt tối ưu lý thuyết nhờ kỹ thuật mở rộng bao đóng. Các hạn chế mà FP-Growth để lại bao gồm: vấn đề bộ nhớ khi FP-tree quá lớn, khó song song hóa do chia sẻ nút giữa nhiều giao dịch, thứ tự sắp xếp items không luôn tối ưu, và cần biến thể riêng cho khai thác tập đóng hay tập tối đại.

2. Ví dụ minh họa tay

2.1 Ví dụ 1: Trường hợp cơ sở

2.1.1 Cơ sở dữ liệu gốc

Xét cơ sở dữ liệu giao dịch $\mathcal{D}$ gồm 6 giao dịch trên itemset $\mathcal{I} = \{A, B, C, D, E\}$ với minimum support threshold $\xi = 3$. Cơ sở dữ liệu này được trình bày trong Bảng 2.1.

Table 2.1: Cơ sở dữ liệu giao dịch cho ví dụ 1

TID	Các items
T_1	$\{A, B, D, E\}$
T_2	$\{B, C, E\}$
T_3	$\{A, B, D, E\}$
T_4	$\{A, B, C, E\}$
T_5	$\{A, B, C, D, E\}$
T_6	$\{B, C, D\}$

2.1.2 Bước 1: Quét lần 1 — Tìm items phổ biến cấp một

Kết quả đếm số lần xuất hiện của mỗi items được trình bày trong Bảng 2.2.

Table 2.2: support các items

Mục	A	B	C	D	E
support	4	6	4	4	5

Tất cả 5 items đều có support ≥ 3 , nên tất cả đều phổ biến. Sắp xếp theo support giảm dần: $\langle B : 6, E : 5, A : 4, C : 4, D : 4 \rangle$. Với A, C, D cùng có support bằng 4, ta chọn thứ tự bằng chữ cái: A trước C trước D .

$$\text{FList} = \langle B, E, A, C, D \rangle.$$

2.1.3 Bước 2: Quét lần 2 — Xây dựng FP-tree

Sau khi sắp xếp lại các items phổ biến trong mỗi giao dịch theo FList, ta thu được Bảng 2.3.

Table 2.3: Giao dịch sau khi sắp xếp theo FList

TID	Frequent item đã sắp xếp
T_1	$\langle B, E, A, D \rangle$
T_2	$\langle B, E, C \rangle$
T_3	$\langle B, E, A, D \rangle$
T_4	$\langle B, E, A, C \rangle$
T_5	$\langle B, E, A, C, D \rangle$
T_6	$\langle B, C, D \rangle$

Chèn lần lượt từng giao dịch vào FP-tree:

Chèn $T_1 = \langle B, E, A, D \rangle$: Tạo đường mới từ gốc: $(B : 1) \rightarrow (E : 1) \rightarrow (A : 1) \rightarrow (D : 1)$.

Chèn $T_2 = \langle B, E, C \rangle$: Chia sẻ tiền tố $\langle B, E \rangle$ với T_1 . Tăng bộ đếm: $(B : 2), (E : 2)$. Tạo nút mới $(C : 1)$ là con của $(E : 2)$.

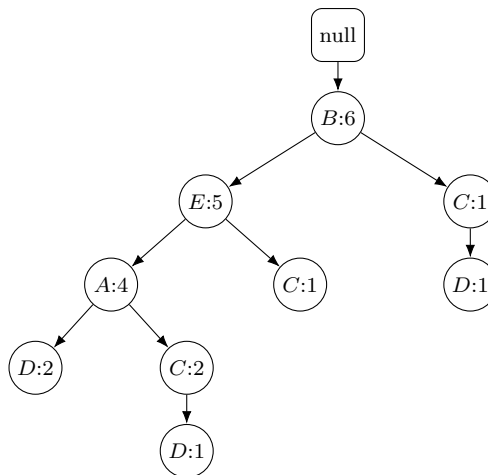
Chèn $T_3 = \langle B, E, A, D \rangle$: Trùng hoàn toàn với đường của T_1 . Tăng bộ đếm: $(B : 3), (E : 3), (A : 2), (D : 2)$.

Chèn $T_4 = \langle B, E, A, C \rangle$: Chia sẻ tiền tố $\langle B, E, A \rangle$. Tăng bộ đếm: $(B : 4), (E : 4), (A : 3)$. Tạo nút mới $(C : 1)$ là con của $(A : 3)$.

Chèn $T_5 = \langle B, E, A, C, D \rangle$: Chia sẻ tiền tố $\langle B, E, A, C \rangle$ (nút C con của A). Tăng bộ đếm: $(B : 5), (E : 5), (A : 4), (C : 2)$. Tạo nút mới $(D : 1)$ là con của $(C : 2)$.

Chèn $T_6 = \langle B, C, D \rangle$: Chia sẻ chỉ nút B với cây hiện tại. Tăng bộ đếm: $(B : 6)$. Tạo đường mới: $(C : 1) \rightarrow (D : 1)$ là con của $(B : 6)$.

FP-tree hoàn chỉnh được minh họa trong Hình 2.1.

**Figure 2.1:** FP-tree thu được sau khi chèn toàn bộ giao dịch của Ví dụ 1

Bảng 2.4 trình bày header table tương ứng với FP-tree trong Hình 2.1.

Table 2.4: Header table của FP-tree trong Ví dụ 1

Mục	support
B	6
E	5
A	4
C	4
D	4

2.1.4 Bước 3: Khai thác FP-tree — Xây dựng conditional pattern-base

Duyệt header table từ items có support thấp nhất (D) đến cao nhất (B).

Khai thác items D (sup = 4)

Tìm tất cả nút D trong FP-tree qua node-link. Có 3 nút D :

- Nút $D : 2$ — đường: $\text{null} \rightarrow B \rightarrow E \rightarrow A \rightarrow D$. Tiền tố đã biến đổi: $\langle B : 2, E : 2, A : 2 \rangle$.
- Nút $D : 1$ — đường: $\text{null} \rightarrow B \rightarrow E \rightarrow A \rightarrow C \rightarrow D$. Tiền tố đã biến đổi: $\langle B : 1, E : 1, A : 1, C : 1 \rangle$.
- Nút $D : 1$ — đường: $\text{null} \rightarrow B \rightarrow C \rightarrow D$. Tiền tố đã biến đổi: $\langle B : 1, C : 1 \rangle$.

Conditional pattern-base của D : $\{(BEA : 2), (BEAC : 1), (BC : 1)\}$.

Đếm items trong conditional pattern-base: $B : 4, E : 3, A : 3, C : 2$. Với ngưỡng $\xi = 3$: items C bị loại ($\text{sup} = 2 < 3$). Các items phổ biến trong conditional pattern-base: $B : 4, E : 3, A : 3$.

Conditional FP-tree | D : đường đơn $\langle B : 4 \rightarrow E : 3 \rightarrow A : 3 \rangle$.

Vì đây là đường đơn, ta liệt kê tất cả tổ hợp rồi nối với D . Kết quả được tóm tắt trong Bảng 2.5:

Table 2.5: Frequent itemset từ conditional FP-tree | D

Tổ hợp từ đường đơn	Frequent itemset (nối với D)
$\{B\}$ sup=4	$\{B, D\}$ sup=4
$\{E\}$ sup=3	$\{E, D\}$ sup=3
$\{A\}$ sup=3	$\{A, D\}$ sup=3
$\{B, E\}$ sup=3	$\{B, E, D\}$ sup=3
$\{B, A\}$ sup=3	$\{B, A, D\}$ sup=3
$\{E, A\}$ sup=3	$\{E, A, D\}$ sup=3
$\{B, E, A\}$ sup=3	$\{B, E, A, D\}$ sup=3

Frequent itemset chứa D : $\{D : 4, BD : 4, ED : 3, AD : 3, BED : 3, BAD : 3, EAD : 3, BEAD : 3\}$.

Khai thác items C (sup = 4)

Các nút C trong FP-tree:

- Nút $C : 2$ — đường: $\text{null} \rightarrow B \rightarrow E \rightarrow A \rightarrow C$. Tiền tố: $\langle B : 2, E : 2, A : 2 \rangle$.
- Nút $C : 1$ — đường: $\text{null} \rightarrow B \rightarrow E \rightarrow C$. Tiền tố: $\langle B : 1, E : 1 \rangle$.
- Nút $C : 1$ — đường: $\text{null} \rightarrow B \rightarrow C$. Tiền tố: $\langle B : 1 \rangle$.

Conditional pattern-base của C : $\{(BEA : 2), (BE : 1), (B : 1)\}$.

Đếm: $B : 4, E : 3, A : 2$. Item A bị loại. Conditional FP-tree | C : $\langle B : 4 \rightarrow E : 3 \rangle$.

Đường đơn, liệt kê tổ hợp nối với C :

Frequent itemset chứa C : $\{C : 4, BC : 4, EC : 3, BEC : 3\}$.

Khai thác items A (sup = 4)

Nút $A : 4$ duy nhất — đường: $\text{null} \rightarrow B \rightarrow E \rightarrow A$. Tiền tố: $\langle B : 4, E : 4 \rangle$.

Conditional pattern-base của A : $\{(BE : 4)\}$.

Đếm: $B : 4, E : 4$. Cả hai phổ biến. Conditional FP-tree | A : $\langle B : 4 \rightarrow E : 4 \rangle$.

Đường đơn, liệt kê:

Frequent itemset chứa A : $\{A : 4, BA : 4, EA : 4, BEA : 4\}$.

Khai thác items E (sup = 5)

Nút $E : 5$ duy nhất — đường: $\text{null} \rightarrow B \rightarrow E$. Tiền tố: $\langle B : 5 \rangle$.

Conditional pattern-base của E : $\{(B : 5)\}$.

Conditional FP-tree | E : $\langle B : 5 \rangle$ — đường đơn một nút.

Frequent itemset chứa E : $\{E : 5, BE : 5\}$.

Khai thác items B (sup = 6)

Nút B : 6 duy nhất là con trực tiếp của gốc. Tiền tố rỗng.

Conditional pattern-base rỗng. Không có conditional FP-tree.

Frequent itemset chứa B : $\{B : 6\}$.

2.1.5 Bước 4: Tổng hợp kết quả

Table 2.6: Tổng hợp tất cả frequent itemset

Itemset	Sup	Itemset	Sup
$\{B\}$	6	$\{B, E, D\}$	3
$\{E\}$	5	$\{B, A, D\}$	3
$\{A\}$	4	$\{E, A, D\}$	3
$\{C\}$	4	$\{B, E, C\}$	3
$\{D\}$	4	$\{B, E, A\}$	4
$\{B, E\}$	5	$\{B, E, A, D\}$	3
$\{B, A\}$	4		
$\{B, C\}$	4		
$\{B, D\}$	4		
$\{E, A\}$	4		
$\{E, C\}$	3		
$\{E, D\}$	3		
$\{A, D\}$	3		

Tổng hợp cuối cùng trong Bảng 2.6 cho thấy có **18 frequent itemset**.

2.1.6 Bước 5: Kiểm tra chéo

Để kiểm chứng tính đúng đắn, ta liệt kê thủ công tất cả itemset và đếm trực tiếp từ cơ sở dữ liệu gốc.

Kiểm tra 1-itemset: A xuất hiện trong T_1, T_3, T_4, T_5 nên $\text{sup}(A) = 4$. B xuất hiện trong $T_1, T_2, T_3, T_4, T_5, T_6$ nên $\text{sup}(B) = 6$. C xuất hiện trong T_2, T_4, T_5, T_6 nên

$\text{sup}(C) = 4$. D xuất hiện trong T_1, T_3, T_5, T_6 nên $\text{sup}(D) = 4$. E xuất hiện trong T_1, T_2, T_3, T_4, T_5 nên $\text{sup}(E) = 5$.

Kiểm tra một số 2-itemset: $\{A, C\}$: xuất hiện trong T_4, T_5 nên $\text{sup} = 2 < 3$ — kết luận không phổ biến là chính xác. $\{C, D\}$: xuất hiện trong T_5, T_6 nên $\text{sup} = 2 < 3$ — kết luận không phổ biến là chính xác. $\{B, E\}$: xuất hiện trong T_1, T_2, T_3, T_4, T_5 nên $\text{sup} = 5$. $\{B, D\}$: xuất hiện trong T_1, T_3, T_5, T_6 nên $\text{sup} = 4$.

Kiểm tra 3-itemset: $\{B, E, A\}$: xuất hiện trong T_1, T_3, T_4, T_5 nên $\text{sup} = 4$. $\{B, E, A, D\}$: xuất hiện trong T_1, T_3, T_5 nên $\text{sup} = 3$. $\{A, C, D\}$: chỉ xuất hiện trong T_5 nên $\text{sup} = 1 < 3$ — kết luận không phổ biến là chính xác.

Kiểm tra itemset 5 phần tử: $\{A, B, C, D, E\}$: chỉ xuất hiện trong T_5 nên $\text{sup} = 1 < 3$ — kết luận không phổ biến là chính xác.

Kết quả kiểm tra chéo xác nhận: thuật toán FP-Growth cho kết quả đầy đủ và chính xác.

2.2 Ví dụ 2: Tình huống đặc biệt — FP-tree đường đơn

2.2.1 Mô tả tình huống đặc biệt

Tình huống đặc biệt quan trọng trong FP-Growth là khi FP-tree chỉ gồm **một đường đơn duy nhất** — tức cây không có bất kỳ nút phân nhánh nào. Đây là trường hợp xảy ra khi tất cả các giao dịch, sau khi loại bỏ items không phổ biến và sắp xếp, đều là tiền tố của cùng một chuỗi.

Tình huống này quan trọng vì hai lý do. Thứ nhất, nó cho phép tối ưu hóa đáng kể: thay vì đệ quy xây conditional FP-tree nhiều mức, ta chỉ cần liệt kê tất cả tổ hợp con các nút trên đường với support bằng giá trị nhỏ nhất trong tổ hợp. Thứ hai, đây cũng là trường hợp mà frequent itemset đóng trùng khớp hoàn toàn với frequent itemset tối đại, tạo nên một mối liên hệ lý thú giữa các khái niệm lý thuyết.

2.2.2 Cơ sở dữ liệu gốc

Xét cơ sở dữ liệu $\mathcal{D}$ gồm 7 giao dịch trên $\mathcal{I} = \{P, Q, R, S, T\}$ với $\xi = 2$. Cơ sở dữ liệu này được trình bày trong Bảng 2.7.

Table 2.7: Cơ sở dữ liệu giao dịch cho ví dụ 2

TID	Các items
T_1	$\{P, Q, R, S, T\}$
T_2	$\{P, Q, R, S, T\}$
T_3	$\{P, Q, R, S\}$
T_4	$\{P, Q, R\}$
T_5	$\{P, Q, R\}$
T_6	$\{P, Q\}$
T_7	$\{P\}$

Cơ sở dữ liệu này được thiết kế có chủ đích: mỗi giao dịch là tiền tố của giao dịch dài nhất, đảm bảo FP-tree sẽ chỉ có một đường đơn.

2.2.3 Bước 1: Quét lần 1 — Tìm items phổ biến cấp một

Table 2.8: support các items

Mục	P	Q	R	S	T
support	7	6	5	3	2

Bảng 2.8 cho thấy tất cả 5 items có support ≥ 2 , nên đều phổ biến. Sắp xếp theo support giảm dần:

$$FList = \langle P : 7, Q : 6, R : 5, S : 3, T : 2 \rangle.$$

2.2.4 Bước 2: Quét lần 2 — Xây dựng FP-tree

Sau khi sắp xếp lại mỗi giao dịch theo FList, ta thu được Bảng 2.9:

Table 2.9: Giao dịch sau khi sắp xếp

TID	Item đã sắp xếp
T_1	$\langle P, Q, R, S, T \rangle$
T_2	$\langle P, Q, R, S, T \rangle$
T_3	$\langle P, Q, R, S \rangle$
T_4	$\langle P, Q, R \rangle$
T_5	$\langle P, Q, R \rangle$
T_6	$\langle P, Q \rangle$
T_7	$\langle P \rangle$

Chèn lần lượt:

Chèn T_1 : Tạo đường: $(P : 1) \rightarrow (Q : 1) \rightarrow (R : 1) \rightarrow (S : 1) \rightarrow (T : 1)$.

Chèn T_2 : Trùng hoàn toàn với T_1 . Tăng toàn bộ: $(P : 2) \rightarrow (Q : 2) \rightarrow (R : 2) \rightarrow (S : 2) \rightarrow (T : 2)$.

Chèn $T_3 = \langle P, Q, R, S \rangle$: Chia sẻ tiền tố $\langle P, Q, R, S \rangle$ — tất cả 4 nút đã tồn tại. Tăng bộ đếm: $(P : 3), (Q : 3), (R : 3), (S : 3)$.

Chèn $T_4 = \langle P, Q, R \rangle$: Chia sẻ tiền tố $\langle P, Q, R \rangle$. Tăng: $(P : 4), (Q : 4), (R : 4)$.

Chèn $T_5 = \langle P, Q, R \rangle$: Giống T_4 . Tăng: $(P : 5), (Q : 5), (R : 5)$.

Chèn $T_6 = \langle P, Q \rangle$: Chia sẻ tiền tố $\langle P, Q \rangle$. Tăng: $(P : 6), (Q : 6)$.

Chèn $T_7 = \langle P \rangle$: Chia sẻ nút P . Tăng: $(P : 7)$.

FP-tree hoàn chỉnh, là một đường đơn duy nhất, được minh họa trong Hình 2.2.

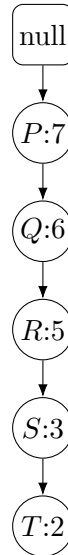


Figure 2.2: FP-tree dạng đường đơn của Ví dụ 2

Nhận xét quan trọng: FP-tree không có bất kỳ nút phân nhánh nào. Mỗi nút chỉ có đúng một nút con. Đây chính là trường hợp đường đơn.

2.2.5 Bước 3: Khai thác — Áp dụng tối ưu hóa đường đơn

Theo Bổ đề 3.2 trong bài báo gốc, khi FP-tree gồm một đường đơn $\langle a_1 : s_1 \rightarrow a_2 : s_2 \rightarrow \dots \rightarrow a_k : s_k \rangle$, tập đầy đủ các frequent itemset được sinh bằng cách liệt kê tất cả tổ hợp con không rỗng, với support của mỗi tổ hợp bằng giá trị nhỏ nhất của các bộ đếm trong tổ hợp đó.

Đường đơn: $\langle P : 7 \rightarrow Q : 6 \rightarrow R : 5 \rightarrow S : 3 \rightarrow T : 2 \rangle$.

Việc liệt kê $2^5 - 1 = 31$ tổ hợp con được tổng hợp trong Bảng 2.10:

Table 2.10: Tất cả frequent itemset — liệt kê từ đường đơn

Itemset	Sup	Itemset	Sup	Itemset	Sup
$\{P\}$	7	$\{P, Q, R\}$	5	$\{P, Q, R, S, T\}$	2
$\{Q\}$	6	$\{P, Q, S\}$	3	$\{P, Q, R, S\}$	3
$\{R\}$	5	$\{P, Q, T\}$	2	$\{P, Q, R, T\}$	2
$\{S\}$	3	$\{P, R, S\}$	3	$\{P, Q, S, T\}$	2
$\{T\}$	2	$\{P, R, T\}$	2	$\{P, R, S, T\}$	2
$\{P, Q\}$	6	$\{P, S, T\}$	2	$\{Q, R, S, T\}$	2
$\{P, R\}$	5	$\{Q, R, S\}$	3		
$\{P, S\}$	3	$\{Q, R, T\}$	2		
$\{P, T\}$	2	$\{Q, S, T\}$	2		
$\{Q, R\}$	5	$\{R, S, T\}$	2		
$\{Q, S\}$	3	$\{P, Q, R, T\}$	2		
$\{Q, T\}$	2				
$\{R, S\}$	3				
$\{R, T\}$	2				
$\{S, T\}$	2				

Tổng cộng: **31 frequent itemset**.

2.2.6 Bước 4: Phân tích tình huống đặc biệt

Tại sao đây là trường hợp đặc biệt quan trọng

Về mặt thuật toán: Nếu không nhận diện đường đơn, thuật toán sẽ thực hiện đệ quy 5 mức: khai thác T , xây conditional FP-tree $|T$ (lại đường đơn $\langle P : 2 \rightarrow Q : 2 \rightarrow R : 2 \rightarrow S : 2 \rangle$), rồi lại đệ quy trong đó. Tổng cộng phải xây $5 + 4 + 3 + 2 + 1 = 15$ conditional FP-tree. Với tối ưu hóa đường đơn, ta chỉ cần một bước liệt kê tổ hợp, tiết kiệm toàn bộ chi phí đệ quy.

Về mặt lý thuyết — Tập đóng trùng với tập tối đại: Xét các giá trị support xuất hiện: $\{7, 6, 5, 3, 2\}$. Bảng 2.11 cho thấy với mỗi giá trị support, frequent itemset đóng là itemset lớn nhất có đúng support đó:

Table 2.11: Tập đóng và tập tối đại

Tập đóng	Sup	Tối đại?
$\{P\}$	7	Không (có tập cha $\{P, Q\}$ phổ biến)
$\{P, Q\}$	6	Không (có tập cha $\{P, Q, R\}$ phổ biến)
$\{P, Q, R\}$	5	Không (có tập cha $\{P, Q, R, S\}$ phổ biến)
$\{P, Q, R, S\}$	3	Không (có tập cha $\{P, Q, R, S, T\}$ phổ biến)
$\{P, Q, R, S, T\}$	2	Có

Kiểm tra tính đóng: Với $\{P\}$ sup=7, ta có $\{P, Q\}$ sup=6 $\neq$ 7, nên không có tập cha cùng support, vậy $\{P\}$ là đóng. Tương tự cho tất cả itemset trên.

Kiểm tra tính tối đại: Chỉ có $\{P, Q, R, S, T\}$ là frequent itemset tối đại duy nhất vì đây là frequent itemset lớn nhất, mọi tập cha đều không phổ biến (vì $\mathcal{I}$ chỉ có 5 items).

Trong trường hợp này, $|\mathcal{M}| = 1$ và $|\mathcal{C}| = 5$ và $|\mathcal{F}| = 31$. Đây minh họa rõ ràng quan hệ $\mathcal{M} \subset \mathcal{C} \subset \mathcal{F}$ với sự chênh lệch kích thước rất lớn.

Kiểm tra chéo

Kiểm tra một số itemset bằng cách đếm trực tiếp:

$\{Q, R, S, T\}$: xuất hiện trong T_1, T_2 (là hai giao dịch chứa đủ cả Q, R, S, T) nên sup = 2.

$\{P, R, S\}$: xuất hiện trong T_1, T_2, T_3 (ba giao dịch chứa cả P, R, S) nên sup = 3.

$\{P, Q, R, S, T\}$: chỉ T_1, T_2 chứa tất cả 5 items, sup = 2.

$\{P, Q, R\}$: T_1, T_2, T_3, T_4, T_5 chứa cả P, Q, R , sup = 5.

So sánh chi phí

Bảng 2.12 so sánh trực tiếp chi phí giữa cách khai thác thông thường và cách khai thác có tối ưu hóa đường đơn.

Table 2.12: So sánh chi phí giữa hai cách khai thác đường đơn

Phương pháp	Số FP-tree xây dựng	Số bước đệ quy
Không tối ưu (đệ quy thông thường)	15	15
Có tối ưu (liệt kê tổ hợp)	0	0

Kết luận: Tối ưu hóa đường đơn giúp thuật toán FP-Growth xử lý trường hợp này với chi phí $O(2^k)$ chỉ cho phép liệt kê kết quả đầu ra, không tốn thêm chi phí nào

cho việc xây dựng cấu trúc dữ liệu trung gian. Đây là trường hợp FP-Growth đạt hiệu suất tối ưu nhất.

3. Cài đặt thuật toán

Chương này trình bày cách nhóm hiện thực thuật toán FP-Growth bằng Julia, từ tổ chức mã nguồn, cấu trúc dữ liệu, quy trình cài đặt cơ bản đến các kỹ thuật tối ưu hóa và kiểm thử. Cách trình bày trong chương tuân theo các quy ước ở Chương 1 và Chương 2: thuật ngữ kỹ thuật tiếng Anh được chú thích ở lần xuất hiện đầu tiên, các bảng đều có chú thích và được tham chiếu trong văn bản.

3.1 Tổng quan kiến trúc dự án

3.1.1 Môi trường và công cụ

Bảng 3.1 tóm tắt môi trường thực nghiệm và các công cụ được sử dụng để cài đặt, chạy thử và đối chiếu kết quả thuật toán.

Table 3.1: Môi trường và công cụ thực nghiệm

Thành phần	Mô tả
Ngôn ngữ lập trình	Julia 1.12.6
Hệ điều hành	Windows 11
Thư viện phụ thuộc	CSV.jl, DataFrames.jl, Plots.jl, ProgressMeter.jl, Statistics.jl, ArgParse.jl
Công cụ đối chiếu	SPMF 2.42 chạy qua fpgrowth_spmf.jar với JDK 21
Quản lý gói	Project.toml và Manifest.toml, giúp tái lập môi trường chạy

Việc tự cài đặt FP-Growth không sử dụng thư viện khai thác frequent itemset có sẵn giúp nhóm kiểm soát đầy đủ cấu trúc dữ liệu, quy trình đệ quy và các lựa chọn tối ưu hóa ở mức triển khai.

3.1.2 Cấu trúc thư items

Cấu trúc dự án được tổ chức theo hướng tách riêng mã nguồn, dữ liệu, nhật ký thực nghiệm và điểm khởi chạy chương trình. Các thành phần chính được trình bày trong Bảng 3.2.

Table 3.2: Cấu trúc thư items chính của dự án

Đường dẫn	Vai trò
<code>src/</code>	Chứa mã nguồn chính của thuật toán và các hàm hỗ trợ.
<code>src/algorithm/</code>	Chứa hai phiên bản FP-Growth: cơ bản và tối ưu hóa.
<code>data/chuẩn/</code>	Chứa các bộ dữ liệu chuẩn như Connect-4, T10I4D100K và T20I6D100K.
<code>data/analysis/</code>	Chứa dữ liệu thực tế dùng cho phân tích ứng dụng.
<code>data/toy/</code>	Chứa các bộ dữ liệu nhỏ dùng cho kiểm thử đơn vị.
<code>notebooks/</code>	Chứa nhật ký đánh giá, đo hiệu năng và áp dụng thuật toán.
<code>main.jl</code>	Điểm khởi chạy chương trình qua giao diện dòng lệnh.
<code>Project.toml</code> , <code>Manifest.toml</code>	Khai báo và khóa phiên bản các gói phụ thuộc của Julia.

Vai trò của các tệp mã nguồn quan trọng trong thư items `src/` được tóm tắt trong Bảng 3.3. Bảng này giúp liên hệ trực tiếp giữa kiến trúc dự án và các bước của thuật toán FP-Growth đã trình bày ở Chương 1.

Table 3.3: Vai trò của các tệp mã nguồn chính

Tệp mã nguồn	Chức năng chính
<code>FPGrowth.jl</code>	Module cha, xuất các hàm giao tiếp như <code>fpgrowth</code> , <code>fpgrowth_opt</code> , <code>read_spmf</code> và <code>write_spmf</code> .
<code>structures.jl</code>	Định nghĩa nút FP-tree và header table.
<code>algorithm/</code> <code>fpgrowth_base.jl</code>	Cài đặt phiên bản FP-Growth cơ bản theo quy trình hai lượt quét và khai thác đệ quy.
<code>algorithm/</code> <code>fpgrowth_opt.jl</code>	Cài đặt phiên bản tối ưu hóa, tích hợp cắt tĩa đường đơn, lọc bằng mảng bit và tái sử dụng bộ đệm.
<code>utils.jl</code>	Đọc, ghi và chuyển đổi dữ liệu theo định dạng SPMF.
<code>eval.jl</code>	Chạy kiểm thử, đối chiếu SPMF và đo lường hiệu năng.

3.2 Cài đặt cơ bản

Cài đặt cơ bản tập trung vào tính đúng đắn của thuật toán. Phiên bản này bám sát mô tả lý thuyết của FP-Growth: quét cơ sở dữ liệu hai lần, xây dựng FP-tree, tạo conditional pattern-base và khai thác đệ quy bằng pattern growth.

3.2.1 Cấu trúc dữ liệu FP-tree

Trong Julia, nút FP-tree được biểu diễn bằng kiểu `FPNode{T}`, trong đó `T` là kiểu dữ liệu của mã items, thường là `Int`. Đoạn mã trong Listing 3.1 cho thấy mỗi nút lưu tên items, bộ đếm, nút cha, danh sách nút con và liên kết đến nút kế tiếp có cùng tên items.

```
1 mutable struct FPNode{T}
2   item::T
3   count::Int
4   parent::Union{FPNode{T}, Nothing}
5   children::Union{Vector{FPNode{T}}, Nothing}
6   node_link::Union{FPNode{T}, Nothing}
7 end
8
9 mutable struct HeaderTableEntry{T}
10  count::Int
11  head::Union{FPNode{T}, Nothing}
12  tail::Union{FPNode{T}, Nothing}
13 end
```

Listing 3.1: Định nghĩa nút FP-tree và items trong header table

Danh sách nút con được lưu bằng `Vector` thay vì `Dict`. Lựa chọn này làm giảm chi phí bộ nhớ tại các nút sâu, nơi số nhánh thường nhỏ. Ngoài ra, trường `children` được khởi tạo lười, tức chỉ cấp phát khi nút thật sự có con.

3.2.2 Xây dựng FP-tree

Quy trình xây dựng FP-tree gồm hai lượt quét cơ sở dữ liệu:

1. **Lượt quét thứ nhất:** Đếm support của từng items, loại bỏ các items không đạt minimum support threshold `min_support`, sau đó khởi tạo header table.

2. **Lượt quét thứ hai:** Với mỗi giao dịch, lọc các items phổ biến, sắp xếp chúng theo support giảm dần và chèn giao dịch đã sắp xếp vào FP-tree.

Listing 3.2 minh họa hàm chèn một giao dịch vào FP-tree. Hàm này tìm nút con đã tồn tại để tăng bộ đếm; nếu chưa có, thuật toán tạo nút mới và cập nhật liên kết trong header table.

```
1 @inline function _insert_tree!(current, header_table, items, count)
2   for item in items
3     child = find_child(current, item)
4     if child !== nothing
5       child.count += count
6     else
7       child = FPNode(item, count, current)
8       push_child!(current, child)
9       link_header!(header_table, item, child)
10    end
11    current = child
12  end
13 end
```

Listing 3.2: Hàm chèn giao dịch đã sắp xếp vào FP-tree

3.2.3 Khai thác đệ quy

Sau khi FP-tree ban đầu được xây dựng, thuật toán khai thác đệ quy bằng pattern growth. Với mỗi items trong header table, thuật toán lần theo các node-link để tạo conditional pattern-base, xây conditional FP-tree và tiếp tục khai thác trên cây con đó.

Quy trình cốt lõi gồm các bước sau:

1. Duyệt các items trong header table theo thứ tự support tăng dần.
2. Tạo mẫu mới bằng cách nối items đang xét vào tiền tố hiện tại.
3. Thu thập các đường tiền tố để xây conditional pattern-base.
4. Xây conditional FP-tree từ các đường tiền tố đủ MinSup.
5. Gọi đệ quy trên conditional FP-tree cho đến khi không còn cây con hợp lệ.

3.2.4 Đọc và ghi dữ liệu theo định dạng SPMF

SPMF là thư viện khai thác mẫu phổ biến được dùng làm công cụ đối chiếu. Định dạng đầu vào của SPMF gồm nhiều dòng giao dịch, trong đó mỗi dòng chứa các mã items dạng số nguyên phân tách bằng khoảng trắng. Đầu ra ghi mỗi frequent itemset kèm support theo dạng `#SUP:.`

Trong cài đặt của nhóm, hàm `read_spmf` đọc dữ liệu và chuyển mỗi dòng thành một `Vector{Int}`. Hàm `write_spmf` sắp xếp kết quả theo độ dài itemset và thứ tự từ điển trước khi ghi tệp, giúp quá trình so sánh với SPMF ổn định và dễ kiểm tra.

3.3 Tối ưu hóa

Phiên bản tối ưu hóa được thiết kế nhằm giảm thời gian chạy và bộ nhớ phân bổ nhưng vẫn giữ nguyên kết quả so với phiên bản cơ bản. Bốn nhóm kỹ thuật chính được tóm tắt trong Bảng 3.4.

Table 3.4: Tóm tắt các kỹ thuật tối ưu hóa

Kỹ thuật	Ý nghĩa triển khai
Cắt tỉa đường đơn	Liệt kê trực tiếp mọi tổ hợp khi FP-tree chỉ còn một đường, tránh xây thêm cây điều kiện.
Lọc bằng mảng bit	Thay tra cứu bằng băm bằng truy cập <code>BitArray</code> để kiểm tra items phổ biến.
Tái sử dụng bộ đệm	Cấp phát trước các mảng tạm và làm rỗng để dùng lại ở mỗi tầng đệ quy.
Ổn định kiểu và nội tuyến hóa	Giúp trình biên dịch Julia suy luận kiểu tốt hơn và giảm chi phí gọi hàm.

3.3.1 Cắt tỉa đường đơn

Cắt tỉa đường đơn được áp dụng khi conditional FP-tree chỉ gồm một đường từ gốc đến lá. Khi đó, thay vì tiếp tục xây các cây điều kiện nhỏ hơn, thuật toán liệt kê trực tiếp tất cả tổ hợp con không rỗng của các items trên đường này. support của mỗi tổ hợp bằng support nhỏ nhất trong các items thuộc tổ hợp.

Kỹ thuật này đặc biệt hiệu quả với dữ liệu dày đặc, nơi các giao dịch chia sẻ nhiều tiền tố và tạo ra các đường đơn dài ở các `MinSup` cao.

3.3.2 Lọc bằng mảng bit

Lọc bằng mảng bit thay thế thao tác tra cứu từ điển bằng phép truy cập trực tiếp vào `bit_filter`. Vì mảng bit được lưu liên tục trong bộ nhớ, thao tác kiểm tra có chi phí hằng số và giảm đáng kể overhead do băm khóa.

3.3.3 Tái sử dụng bộ đệm

Tái sử dụng bộ đệm làm giảm số lần cấp phát đối tượng tạm trong quá trình đệ quy. Các cấu trúc như `item_counts_stack`, `header_table_stack` và `path_buf` được cấp phát trước, sau đó làm rỗng bằng `empty!` để tái sử dụng ở các tầng đệ quy tiếp theo.

Cách làm này làm giảm áp lực lên bộ thu gom rác (Garbage Collection, viết tắt GC) và giúp thời gian chạy ổn định hơn trên các tập dữ liệu lớn.

3.3.4 Ổn định kiểu và nội tuyến hóa

Ổn định kiểu dữ liệu là yêu cầu quan trọng để Julia sinh mã máy hiệu quả. Cấu trúc `FPNode{T}` giúp trình biên dịch biết rõ kiểu của mã items và tránh sử dụng kiểu chung `Any`. Bên cạnh đó, chỉ thị `@inline` được áp dụng cho các hàm nhỏ như `find_child` và `_insert_tree!`, giúp giảm chi phí gọi hàm trong các vòng lặp được thực thi nhiều lần.

3.4 Kiểm thử tự động

Hệ thống kiểm thử tự động được thiết kế để kiểm tra tính đúng đắn của thuật toán trong suốt quá trình phát triển. Bộ kiểm thử gồm năm tệp dữ liệu nhỏ, mỗi tệp đại diện cho một tình huống đặc biệt của FP-Growth. Các kịch bản kiểm thử được mô tả trong Bảng 3.5.

Table 3.5: Các bộ dữ liệu kiểm thử đơn vị

Tệp	Quy mô	Item tiêu kiểm thử
test1.txt	6 giao dịch, 5 items	Kiểm tra các itemset lồng nhau và mức phổ biến giảm dần.
test2.txt	7 giao dịch, 5 items	Kiểm tra dữ liệu mật độ trung bình với nhiều nhánh khác nhau.
test3.txt	6 giao dịch, 4 items	Kiểm tra các tổ hợp phổ biến của nhóm items {2, 3, 4}.
test4.txt	4 giao dịch, 5 items	Kiểm tra tập dữ liệu rất nhỏ và các trường hợp biên.
test5.txt	5 giao dịch, 5 items	Kiểm tra dữ liệu dạng chuỗi, nơi mỗi items chỉ liên kết hạn chế với items kế tiếp.

3.4.1 Quy trình kiểm thử

Quy trình kiểm thử trong hàm `run_unitTest` gồm bốn bước:

1. Thực thi phiên bản FP-Growth tự cài đặt và ghi kết quả ra tệp tạm.
2. Chạy `fpgrowth_spmf.jar` trên cùng dữ liệu và cùng MinSup.
3. So sánh frequent itemset và support giữa hai kết quả.
4. Báo cáo kết quả và dọn dẹp các tệp tạm thời.

Kết quả cho thấy cả hai phiên bản cơ bản và tối ưu đều vượt qua năm bài kiểm thử với tỷ lệ trùng khớp **100%** so với SPMF.

3.5 So sánh phiên bản cơ bản và phiên bản tối ưu

Để đánh giá hiệu quả của các kỹ thuật tối ưu hóa, nhóm đo lường hai phiên bản trên tập dữ liệu dày đặc Connect-4 gồm 67.557 giao dịch, mỗi giao dịch chứa 42 items, tại minimum support threshold $\text{minsup} = 75\%$. Kết quả được trình bày trong Bảng 3.6.

Table 3.6: So sánh hiệu năng giữa phiên bản cơ bản và phiên bản tối ưu hóa

Tiêu chí đánh giá	Phiên bản cơ bản	Phiên bản tối ưu
Kết quả đúng đắn khi đối chiếu SPMF	100% khớp	100% khớp
Thời gian thực thi trung bình	2,968 giây	2,293 giây
Tỷ lệ cải thiện tốc độ	—	Nhanh hơn 22,74%
Bộ nhớ phân bổ tối đa	774,13 MB	627,81 MB
Tỷ lệ tiết kiệm bộ nhớ	—	Giảm 18,90%
Số kỹ thuật tối ưu tích hợp	0	4

Kết quả cho thấy phiên bản tối ưu hóa rút ngắn thời gian thực thi và giảm bộ nhớ phân bổ. Cụ thể, thời gian chạy giảm từ 2,968 giây xuống 2,293 giây, trong khi bộ nhớ phân bổ giảm từ 774,13 MB xuống 627,81 MB. Điều này cho thấy các tối ưu hóa ở mức cấu trúc dữ liệu và cấp phát bộ nhớ có tác động thực tế rõ rệt.

3.6 Giao diện dòng lệnh

Chương trình hỗ trợ giao diện dòng lệnh thông qua tệp `main.jl`. Các tham số chính được mô tả trong Bảng 3.7.

Table 3.7: Các tham số dòng lệnh của chương trình

Tham số	Tên ngắn	Ý nghĩa
<code>-input</code>	<code>-i</code>	Đường dẫn tới tệp dữ liệu đầu vào theo định dạng SPMF.
<code>-minsup</code>	<code>-s</code>	Minimum support threshold, ở dạng tỷ lệ $0 < s \leq 1$ hoặc số nguyên tuyệt đối.
<code>-output</code>	<code>-o</code>	Đường dẫn tệp kết quả đầu ra; mặc định là <code>results/output.txt</code> .

Ví dụ lệnh chạy chương trình được trình bày trong Listing 3.3.

```
1 julia --project main.jl --input data/toy/test1.txt --minsup 0.4 --output
  results/result1.txt
```

Listing 3.3: Ví dụ chạy chương trình từ dòng lệnh

Chương trình đọc dữ liệu, chuyển đổi MinSup tương đối sang ngưỡng tuyệt đối nếu cần, thực thi FP-Growth và ghi kết quả ra tệp theo định dạng SPMF.

3.7 Tóm tắt

Chương này đã trình bày quá trình cài đặt thuật toán FP-Growth bằng Julia theo hai phiên bản: cơ bản và tối ưu hóa. Phiên bản cơ bản bảo đảm tính đúng đắn bằng cách bám sát quy trình lý thuyết, còn phiên bản tối ưu cải thiện hiệu năng thông qua cắt tỉa đường đơn, mảng bit, tái sử dụng bộ đệm và ổn định kiểu dữ liệu. Kết quả kiểm thử cho thấy cả hai phiên bản đều khớp 100% với SPMF, trong khi phiên bản tối ưu giúp giảm đáng kể thời gian thực thi và bộ nhớ phân bổ.

4. Thực nghiệm và đánh giá

Chương này trình bày toàn bộ quá trình thực nghiệm nhằm đánh giá tính đúng đắn, hiệu năng và khả năng mở rộng của thuật toán FP-Growth do nhóm tự cài đặt bằng ngôn ngữ Julia. Quá trình đánh giá được thực hiện trên năm tập dữ liệu chuẩn chuẩn, đối chiếu trực tiếp với thư viện SPMF viết bằng Java. Ngoài ra, nhóm cũng phân tích ảnh hưởng của các yếu tố như minimum support threshold, độ dài giao dịch trung bình và kích thước dữ liệu đến hiệu năng thuật toán.

4.1 Tập dữ liệu chuẩn

Nhóm lựa chọn năm tập dữ liệu đa dạng về quy mô, mật độ và nguồn gốc để đảm bảo tính toàn diện của quá trình đánh giá. Bảng 4.1 tóm tắt các đặc trưng chính của từng tập dữ liệu.

Table 4.1: Tổng quan các tập dữ liệu chuẩn

Dataset	Trans.	Số items	Avg. length	Đặc điểm
Connect-4	67.557	126	42	Dày đặc
T10I4D100K	100.000	~870	10	Thưa, tổng hợp
T20I6D100K	99.910	893	20	Thưa, tổng hợp
Accidents	340.183	468	33,8	Dày đặc, quy mô lớn
Retail	88.162	16.470	10,3	Thưa, thực tế

4.1.1 Connect-4

Tập dữ liệu Connect-4 được cung cấp bởi UCI Machine Learning Repository, chứa 67.557 trạng thái hợp lệ của bàn cờ Connect-4 tại thời điểm sau 8 nước đi. Mỗi giao dịch gồm 42 thuộc tính phân loại biểu diễn trạng thái các ô trên bàn cờ kích thước 7×6 , sau khi mã hóa mã hóa nhị phân tạo ra 126 items. Đây là tập dữ liệu dày đặc điển hình: mỗi giao dịch luôn có độ dài cố định 42, các item có tần suất đồng xuất hiện cao, tạo ra nhiều frequent itemset dài khi giảm ngưỡng support. Connect-4 là điều kiện lý tưởng để kiểm chứng hiệu quả của kỹ thuật Single Path Pruning trên các nhánh đơn dài trong FP-tree.

4.1.2 T10I4D100K

T10I4D100K là tập dữ liệu tổng hợp được tạo bởi IBM Almaden Quest Research Group nhằm mô phỏng hành vi mua sắm bán lẻ. Tập dữ liệu gồm 100.000 giao dịch với khoảng 870 items, độ dài giao dịch trung bình là 10 và kích thước trung bình của frequent itemset tối đại tiềm năng là 4. Đây là tập dữ liệu thừa điển hình: mỗi giao dịch chỉ chứa một phần nhỏ trong tổng số items, phản ánh đúng đặc tính của dữ liệu giao dịch bán lẻ thực tế. Tập này phù hợp để đánh giá khả năng mở rộng và kiểm soát bộ nhớ khi kích thước đầu ra tăng có kiểm soát.

4.1.3 T20I6D100K

T20I6D100K cũng là tập dữ liệu tổng hợp từ IBM Quest Generator, gồm 99.910 giao dịch với 893 items. So với T10I4D100K, tập này có độ dài giao dịch trung bình gấp đôi (20 items) và kích thước frequent itemset tối đại trung bình là 6. Sự gia tăng về độ dài giao dịch làm tăng khả năng đồng xuất hiện giữa các item, dẫn đến FP-tree phân nhánh phức tạp hơn và không gian tìm kiếm rộng hơn. Tập này cho phép đánh giá tác động của độ dài giao dịch đến hiệu năng thuật toán.

4.1.4 Accidents

Tập dữ liệu Accidents được cung cấp bởi Số frequent itemsetMI Repository, chứa 340.183 giao dịch biểu diễn các vụ tai nạn giao thông đã ẩn danh hóa. Mỗi giao dịch gồm tập hợp các thuộc tính mã hóa liên quan đến điều kiện thời tiết, loại phương tiện, tình trạng đường và các yếu tố môi trường. Với 468 items và độ dài giao dịch trung bình 33,8, đây là tập dữ liệu lớn và tương đối dày, sinh ra rất nhiều frequent itemset khi giảm ngưỡng support. Accidents đặt ra thách thức lớn về cả thời gian xử lý lẫn quản lý bộ nhớ cho thuật toán FP-Growth.

4.1.5 Retail

Retail là tập dữ liệu thực tế chứa các giao dịch mua hàng tại một cửa hàng bán lẻ ẩn danh ở Bỉ, được cung cấp bởi Số frequent itemsetMI Repository. Tập dữ liệu gồm 88.162 giao dịch với 16.470 items và độ dài giao dịch trung bình khoảng 10,3. Đặc trưng nổi bật là không gian item cực lớn nhưng mỗi giao dịch ngắn, tạo ra tính thừa cao. Ở các ngưỡng support từ 30% đến 50%, số lượng frequent itemset chỉ dao động từ 1 đến 3, phản ánh đúng thực tế bán lẻ. Retail được chọn làm đối tượng đánh giá khả năng mở rộng do tính chất thừa và quy mô giao dịch phù hợp.

4.2 Kiểm tra tính đúng đắn

Để đảm bảo phiên bản FP-Growth do nhóm cài đặt sinh ra kết quả chính xác, nhóm thực hiện đối chiếu trực tiếp với thư viện SPMF. Quá trình kiểm tra bao gồm hai giai đoạn: kiểm thử đơn vị trên dữ liệu nhỏ và kiểm chứng trên các tập dữ liệu chuẩn.

4.2.1 Kiểm thử đơn vị

Nhóm thiết kế 5 tập dữ liệu nhỏ với các kịch bản khác nhau nhằm kiểm tra từng khía cạnh của thuật toán. Tất cả được chạy với ngưỡng $\text{MinSup} = 40\%$ và đối chiếu kết quả với SPMF.

Table 4.2: Kết quả kiểm thử đơn vị trên 5 tập dữ liệu nhỏ

Test	Item tiêu kiểm thử	Trans.	Số frequent itemset	Kết quả
1	Itemset lồng nhau, tần suất giảm dần	6	19	Đạt
2	Mật độ trung bình, items rải rác	7	15	Đạt
3	Tổ hợp phổ biến nhóm item cụ thể	6	11	Đạt
4	Tập dữ liệu cực nhỏ	5	15	Đạt
5	Dữ liệu dạng chuỗi, liên kết hạn chế	5	5	Đạt
Tỉ lệ đạt				100%

Kết quả cho thấy phiên bản Julia khớp hoàn toàn 100% với SPMF trên cả 5 bài kiểm thử, xác nhận logic cốt lõi của thuật toán hoạt động đúng trước khi tiến hành đánh giá trên dữ liệu lớn.

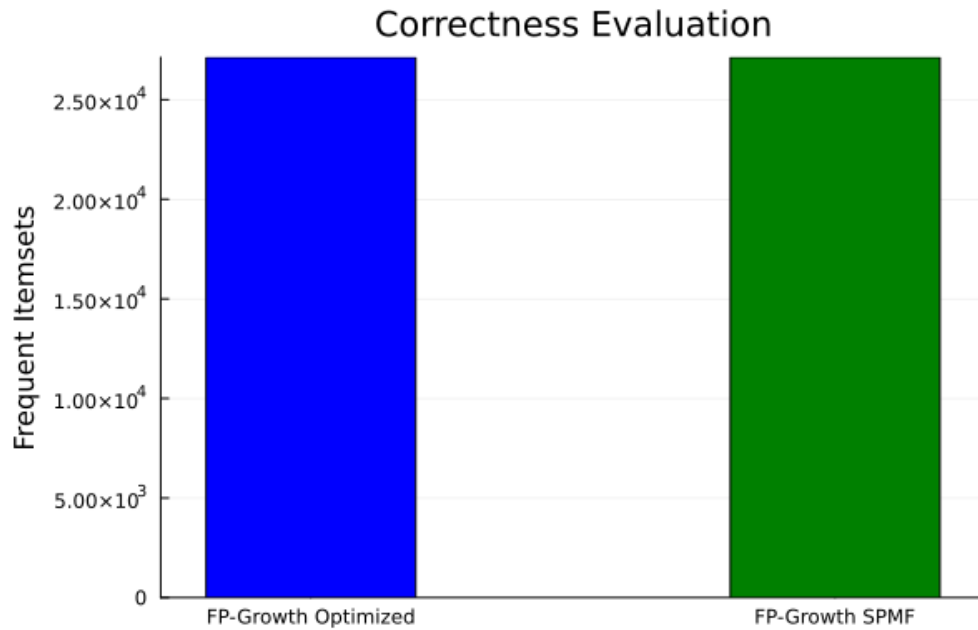
4.2.2 Kiểm chứng trên tập chuẩn

Trên mỗi tập dữ liệu chuẩn, nhóm so sánh số lượng frequent itemset và giá trị support của từng itemset giữa phiên bản FP-Growth tối ưu và SPMF. Bảng 4.3 trình bày kết quả tổng hợp.

Table 4.3: Kết quả kiểm tra tính đúng đắn trên các tập chuẩn

Dataset	MinSup	Optimized itemsets	SPMF itemsets	Match rate
Connect-4	90%	27.127	27.127	100%
T10I4D100K	5%	10	10	100%
T20I6D100K	5%	99	99	100%
Accidents	50%	8.057	8.057	100%
Retail	50%	1	1	100%

Trên toàn bộ năm tập dữ liệu, phiên bản Tối ưu đạt tỉ lệ khớp 100% về cả số lượng itemsets lẫn giá trị absolute support của từng itemset. Kết quả này xác nhận rằng các kỹ thuật tối ưu hóa bao gồm BitArray Filtering, Single Path Pruning và Stack-Buffered Lazy Allocation không làm thay đổi logic đếm support hay điều kiện sinh frequent itemset của thuật toán FP-Growth. Hình 4.1 minh họa kết quả đối chiếu trên tập Connect-4.

**Figure 4.1:** Kết quả kiểm tra tính đúng đắn trên Connect-4 (MinSup = 90%)

4.3 Thời gian chạy theo minsup

Nhóm đo thời gian thực thi của cả ba phiên bản trên từng tập dữ liệu tại nhiều ngưỡng MinSup giảm dần. Mỗi cấu hình được chạy 5 lần và lấy giá trị trung bình nhằm giảm nhiễu đo lường.

4.3.1 Connect-4

Bảng 4.4 trình bày thời gian thực thi của FP-Growth tối ưu và SPMF trên Connect-4 với MinSup từ 75% đến 95%.

Table 4.4: Thời gian thực thi, tính theo giây, trên Connect-4 của phiên bản tối ưu và SPMF

MinSup	Itemsets	Optimized (s)	SPMF (s)
75%	1.585.551	2,258	1,650
80%	533.975	0,813	1,991
85%	142.127	0,225	0,654
90%	27.127	0,145	0,566
95%	2.201	0,131	0,500

Từ 80% đến 95%, phiên bản Tối ưu có thời gian thực thi thấp hơn SPMF. Tại MinSup = 90%, Tối ưu mất khoảng 0,145s trong khi SPMF mất 0,566s. Tuy nhiên, tại MinSup = 75% khi số lượng itemsets lên tới hơn 1,5 triệu, Tối ưu mất 2,258s, cao hơn SPMF (1,650s). Điều này cho thấy chi phí cấp phát bộ nhớ trung gian tăng mạnh khi kích thước đầu ra rất lớn trên dữ liệu dày.

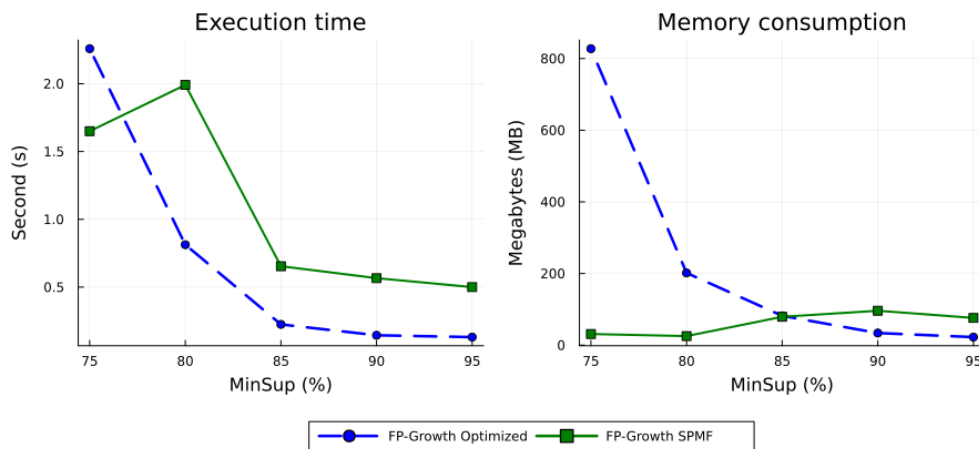


Figure 4.2: So sánh thời gian chạy và bộ nhớ: phiên bản tối ưu và SPMF trên Connect-4

Hình 4.3 trình bày so sánh giữa phiên bản tối ưu và phiên bản cơ bản. Trên toàn bộ dải MinSup, Tối ưu có thời gian thực thi thấp hơn hoặc xấp xỉ Cơ bản. Tại 80%, Tối ưu mất 0,530s so với Cơ bản 0,653s. Lợi ích rõ nhất xuất hiện ở các ngưỡng có kích thước đầu ra lớn, nơi BitArray Filtering giúp giảm chi phí tra cứu và duyệt cấu trúc dữ liệu.

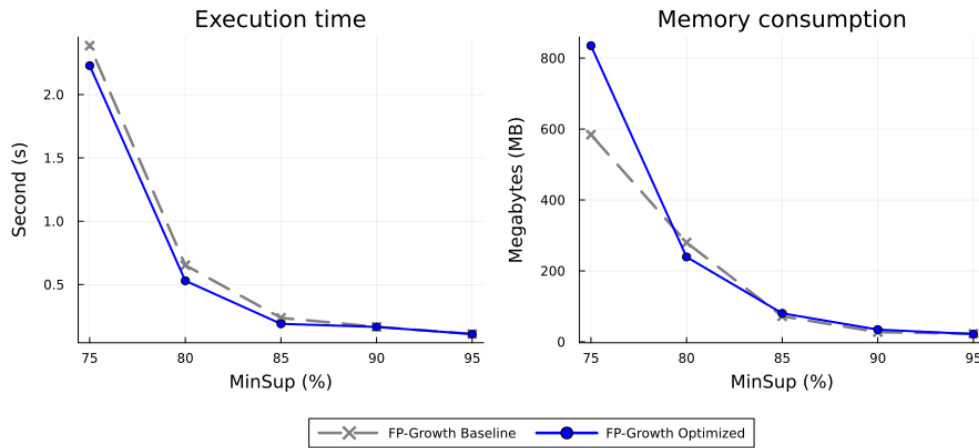


Figure 4.3: So sánh thời gian chạy và bộ nhớ: Tối ưu vs Cơ bản trên Connect-4

4.3.2 T10I4D100K

Table 4.5: Thời gian thực thi, tính theo giây, trên T10I4D100K của phiên bản tối ưu và SPMF

MinSup	Itemsets	Optimized (s)	SPMF (s)
1%	385	0,440	0,967
2%	155	0,141	0,556
3%	60	0,051	0,410
4%	26	0,034	0,408
5%	10	0,022	0,390

Trên tập dữ liệu thưa T10I4D100K, Tối ưu nhanh hơn SPMF ở toàn bộ dải MinSup. Tại MinSup = 1%, Tối ưu mất 0,440s so với 0,967s của SPMF. Tại MinSup = 5%, Tối ưu hoàn thành chỉ trong 0,022s so với 0,390s. Do mỗi giao dịch có độ dài trung bình thấp và số lượng itemsets đầu ra không quá lớn, FP-tree không phát sinh nhiều nhánh phức tạp, giúp các kỹ thuật lọc và cấp phát trong phiên bản Tối ưu phát huy hiệu quả tối đa.

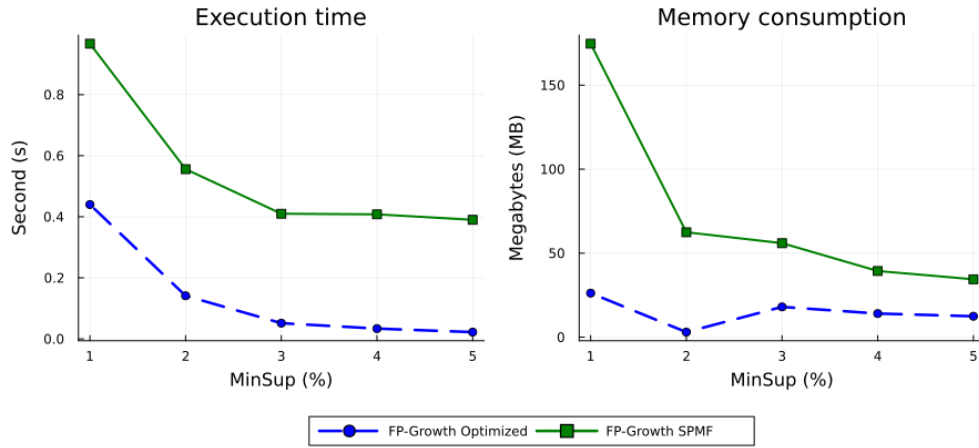


Figure 4.4: So sánh thời gian chạy và bộ nhớ: phiên bản tối ưu và SPMF trên T10I4D100K

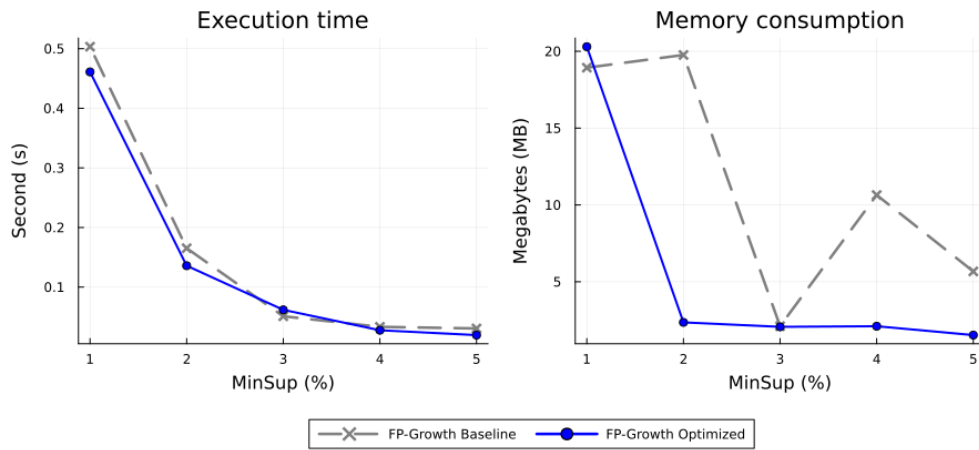


Figure 4.5: So sánh thời gian chạy và bộ nhớ: Tối ưu vs Cơ bản trên T10I4D100K

4.3.3 T20I6D100K

Table 4.6: Thời gian thực thi, tính theo giây, trên T20I6D100K của phiên bản tối ưu và SPMF

MinSup	Itemsets	Optimized (s)	SPMF (s)
1%	1,523	2,329	4,656
2%	378	1,358	2,402
3%	242	0,838	2,174
4%	154	0,549	1,749
5%	99	0,251	1,139

T20I6D100K có độ dài giao dịch trung bình gấp đôi T10I4D100K, dẫn đến không gian tìm kiếm phức tạp hơn. Tối ưu vẫn duy trì lợi thế thời gian trên toàn bộ dải

MinSup: tại 1%, Tối ưu mất 2,329s so với 4,656s của SPMF; tại 5%, Tối ưu mất 0,251s so với 1,139s. Kết quả cho thấy phiên bản Julia thích nghi tốt với dữ liệu giao dịch thưa có độ dài trung bình cao hơn.

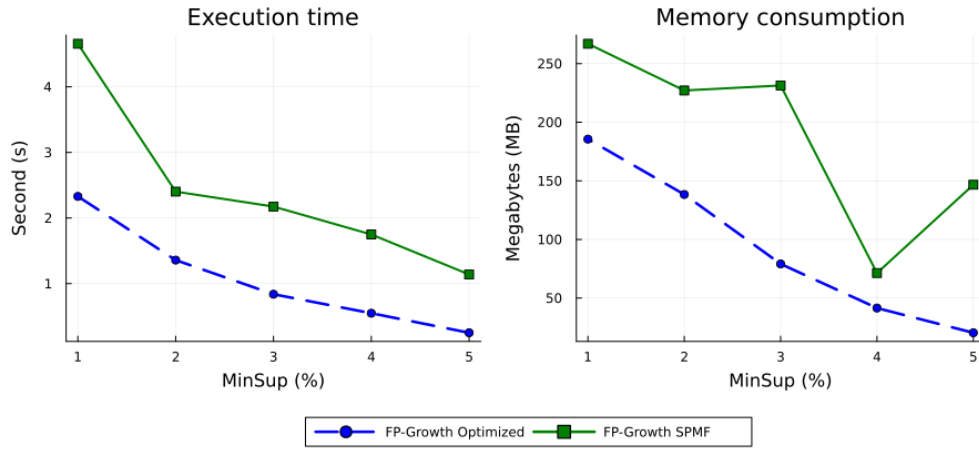


Figure 4.6: So sánh thời gian chạy và bộ nhớ: phiên bản tối ưu và SPMF trên T20I6D100K

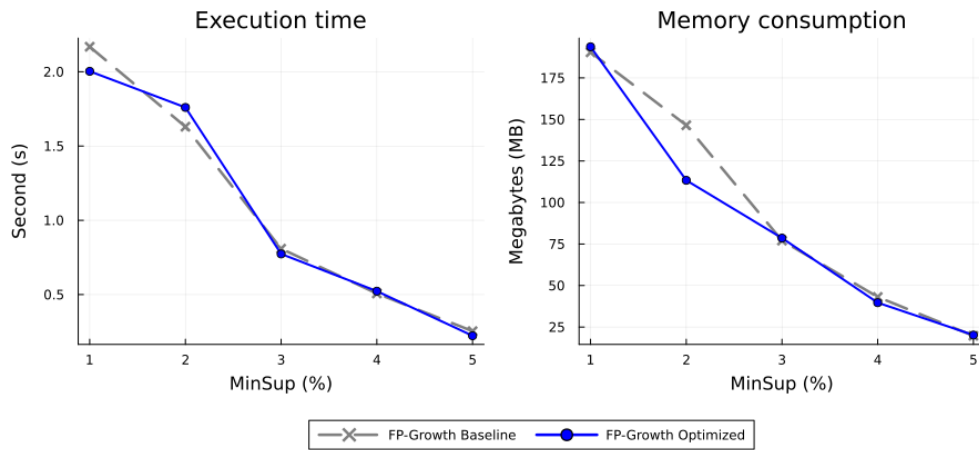


Figure 4.7: So sánh thời gian chạy và bộ nhớ: Tối ưu vs Cơ bản trên T20I6D100K

4.3.4 Accidents

Table 4.7: Thời gian thực thi, tính theo giây, trên Accidents của phiên bản tối ưu và SPMF

MinSup	Itemsets	Optimized (s)	SPMF (s)
30%	149.545	4,007	4,136
35%	68.222	2,589	2,898
40%	32.528	2,337	3,006
45%	16.123	1,376	2,602
50%	8.057	1,074	2,302

Trên tập dữ liệu dày Accidents với hơn 340.000 giao dịch, Tối ưu nhanh hơn hoặc xấp xỉ SPMF ở toàn bộ các ngưỡng. Tại MinSup = 50%, Tối ưu mất 1,074s so với 2,302s của SPMF. Tại MinSup = 30% khi số itemsets lên tới gần 150.000, khoảng cách thu hẹp (4,007s so với 4,136s). Khi MinSup giảm, số lượng itemsets tăng mạnh, làm tăng chi phí khai thác conditional FP-tree, thu hẹp lợi thế thời gian của Tối ưu.

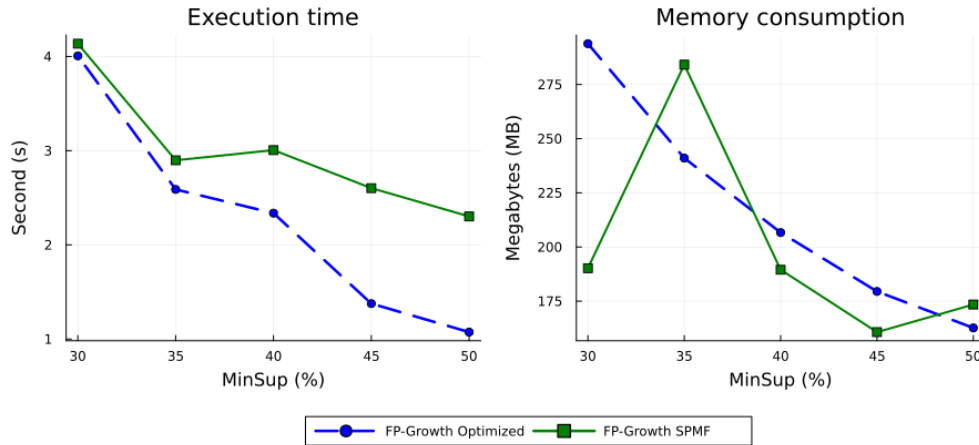


Figure 4.8: So sánh thời gian chạy và bộ nhớ: phiên bản tối ưu và SPMF trên Accidents

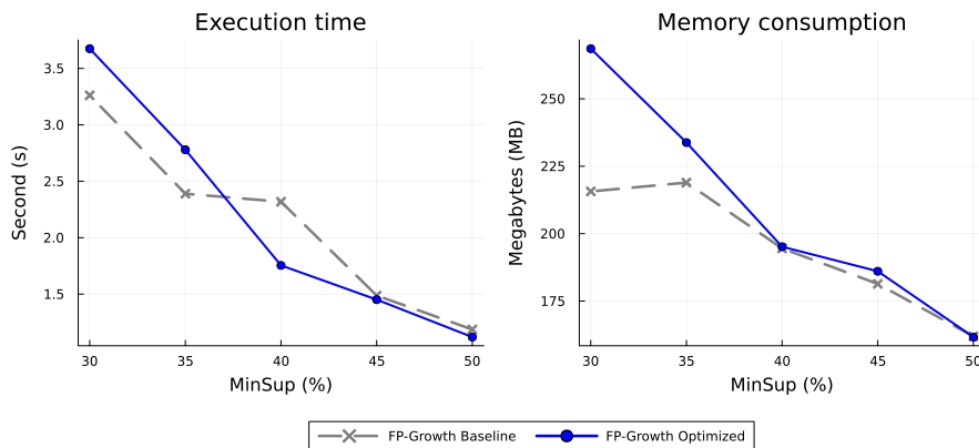


Figure 4.9: So sánh thời gian chạy và bộ nhớ: Tối ưu vs Cơ bản trên Accidents

Đối với so sánh Cơ bản vs Tối ưu trên Accidents, kết quả phức tạp hơn. Tại MinSup thấp (30% và 35%), Cơ bản nhanh hơn Tối ưu do chi phí kiểm tra tối ưu và cấp phát cấu trúc trung gian. Tại các ngưỡng cao hơn (40%, 45%, 50%), Tối ưu cải thiện hoặc tương đương Cơ bản. Điều này cho thấy trên dữ liệu dày, chi phí kiểm tra Single Path Pruning có thể bù trừ lợi ích tối ưu khi FP-tree phân nhánh phức tạp.

4.3.5 Retail

Table 4.8: Thời gian thực thi, tính theo giây, trên Retail của phiên bản tối ưu và SPMF

MinSup	Itemsets	Optimized (s)	SPMF (s)
30%	3	0,027	0,576
35%	2	0,026	0,466
40%	2	0,019	0,537
45%	2	0,038	0,366
50%	1	0,019	0,320

Trên Retail, Tối ưu nhanh hơn SPMF rất rõ rệt, thường từ 10 đến 20 lần. Tối ưu hoàn thành trong khoảng 0,019s đến 0,038s, trong khi SPMF mất 0,320s đến 0,576s. Lợi thế này đến từ đặc trưng dữ liệu rất thưa: số lượng item phổ biến ở ngưỡng 30%–50% cực ít (chỉ 1–3 itemsets), nên Julia không phải khai thác nhiều conditional FP-tree, trong khi SPMF vẫn chịu chi phí nền của JVM.

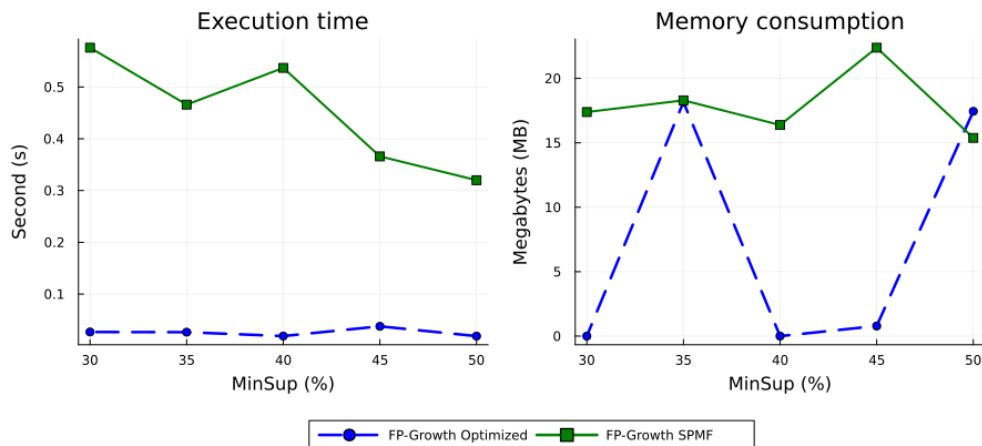


Figure 4.10: So sánh thời gian chạy và bộ nhớ: phiên bản tối ưu và SPMF trên Retail

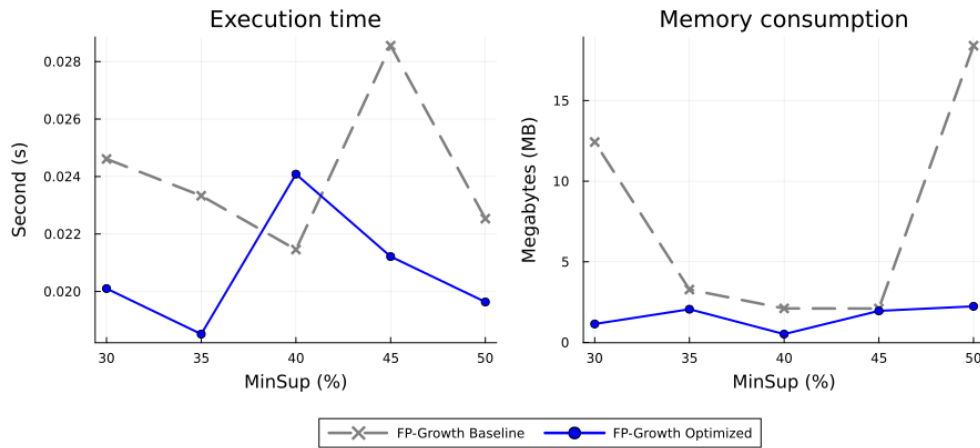


Figure 4.11: So sánh thời gian chạy và bộ nhớ: Tối ưu vs Cơ bản trên Retail

4.4 Số lượng frequent itemset theo minsup

Item này phân tích mối quan hệ giữa ngưỡng MinSup và kích thước tập kết quả trên từng tập dữ liệu, đồng thời so sánh hành vi giữa dữ liệu dày đặc và dữ liệu thưa.

4.4.1 Connect-4

Khi MinSup giảm từ 95% xuống 75%, số lượng frequent itemset trên Connect-4 tăng từ 2.201 lên hơn 1,5 triệu. Trục tung biểu diễn theo thang logarit cho thấy sự tăng trưởng ở nhiều bậc độ lớn. Hiện tượng này phản ánh đặc trưng của tập dữ liệu dày: mật độ đồng xuất hiện cao giữa các item khiến nhiều tổ hợp đồng thời thỏa điều kiện phổ biến khi hạ ngưỡng support.

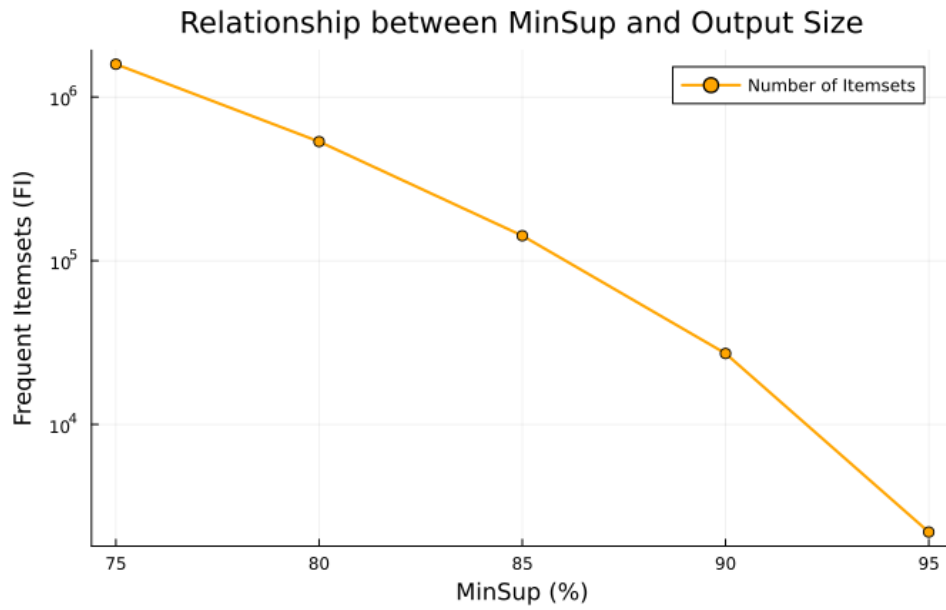


Figure 4.12: Số lượng frequent itemset theo MinSup trên Connect-4

4.4.2 T10I4D100K

Trên T10I4D100K, khi MinSup giảm từ 5% xuống 1%, số lượng frequent itemset tăng từ 10 lên 385. Tốc độ tăng trưởng có kiểm soát, khác biệt rõ rệt so với các tập dữ liệu dày. Dù MinSup giảm xuống 1%, số lượng itemsets vẫn ở mức hàng trăm thay vì hàng trăm nghìn, phản ánh đặc trưng thưa: không gian item lớn nhưng mỗi giao dịch chỉ chứa ít item, khả năng đồng xuất hiện thấp.

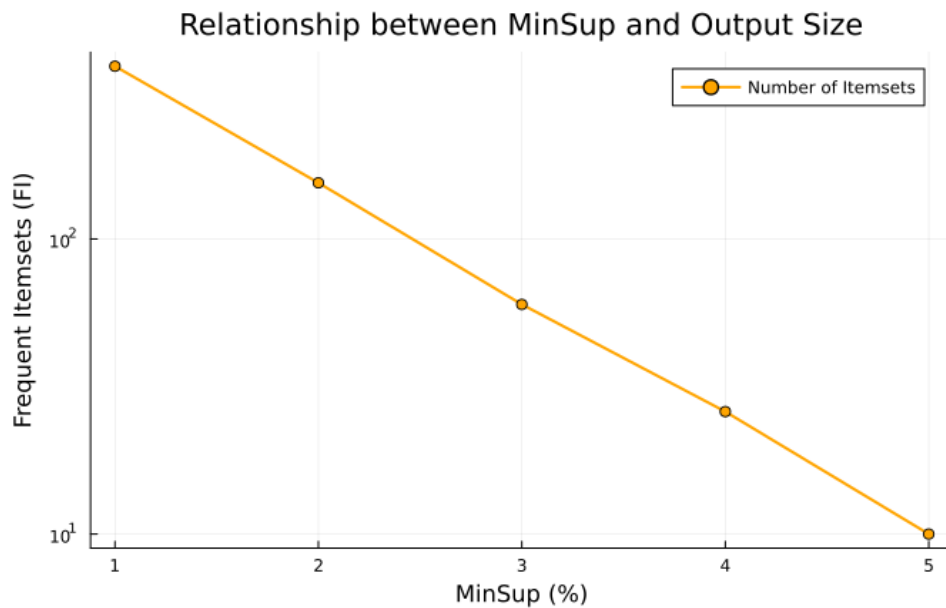


Figure 4.13: Số lượng frequent itemset theo MinSup trên T10I4D100K

4.4.3 T20I6D100K

Khi MinSup giảm từ 5% xuống 1%, số lượng frequent itemset trên T20I6D100K tăng từ 99 lên 1.523, mạnh hơn T10I4D100K do độ dài giao dịch trung bình lớn hơn. Số lượng itemsets tăng khoảng 15 lần khi MinSup giảm từ 5% xuống 1%, cho thấy FP-tree có nhiều nhánh cần khai thác hơn. Tuy nhiên, mức tăng vẫn thấp hơn đáng kể so với các tập dữ liệu dày.

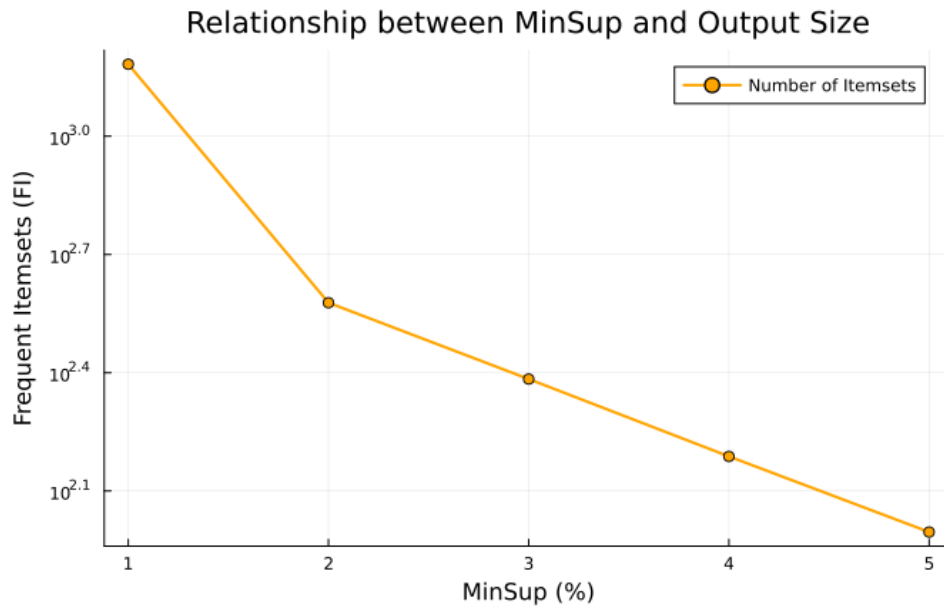


Figure 4.14: Số lượng frequent itemset theo MinSup trên T20I6D100K

4.4.4 Accidents

Trên Accidents, khi MinSup giảm từ 50% xuống 30%, số lượng frequent itemset tăng từ 8.057 lên 149.545. Đây là mức tăng rất mạnh trên thang logarit, phản ánh đặc trưng dày đặc của tập dữ liệu: độ dài giao dịch trung bình cao (33,8) và nhiều item đồng xuất hiện làm tăng số lượng tổ hợp thỏa ngưỡng support.

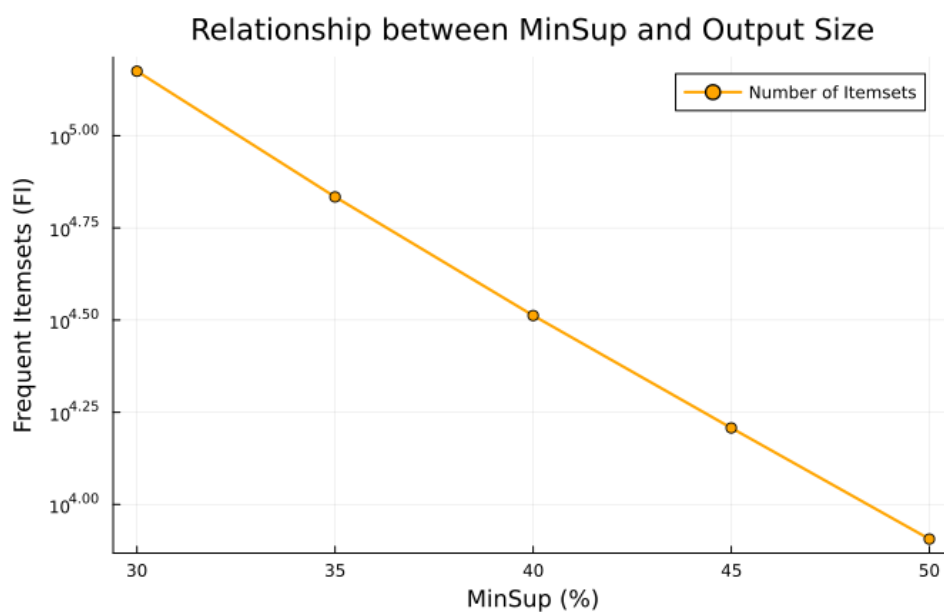


Figure 4.15: Số lượng frequent itemset theo MinSup trên Accidents

4.4.5 Retail

Trên Retail, khi MinSup giảm từ 50% xuống 30%, số lượng frequent itemset chỉ tăng từ 1 lên 3. Quan hệ nghịch biến vẫn tồn tại nhưng độ dốc gần như phẳng. Không xảy ra hiện tượng bùng nổ tổ hợp vì các sản phẩm trong dữ liệu bán lẻ thừa có tần suất đồng xuất hiện rất thấp, ít tổ hợp cùng vượt qua ngưỡng support cao.

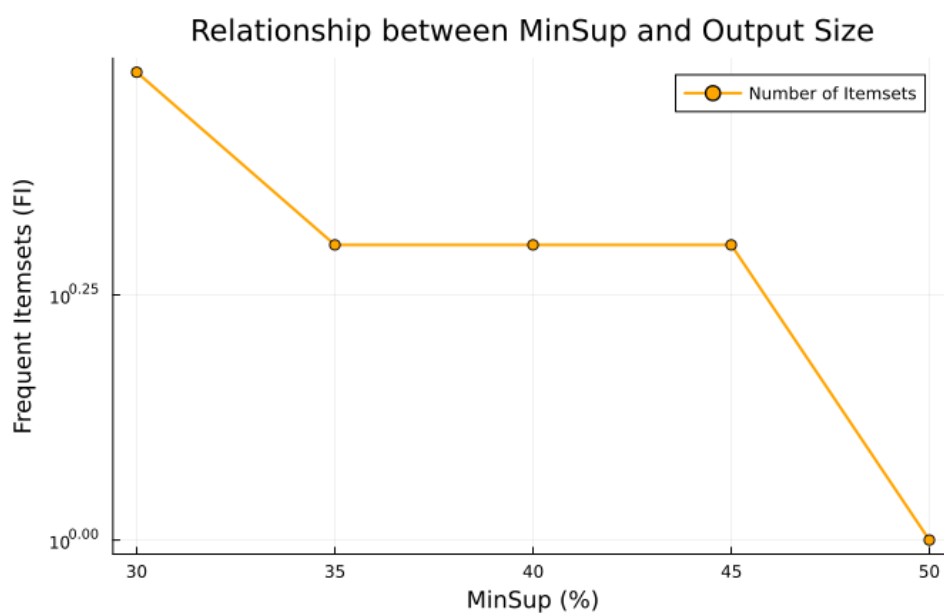


Figure 4.16: Số lượng frequent itemset theo MinSup trên Retail

4.4.6 So sánh giữa dữ liệu dày đặc và dữ liệu thưa

Kết quả thực nghiệm cho thấy sự khác biệt rõ ràng giữa hai nhóm dữ liệu. Trên dữ liệu dày như Connect-4 và Accidents, số lượng frequent itemset tăng theo hàm mũ khi MinSup giảm, dẫn đến bùng nổ tổ hợp và đặt áp lực lớn lên cả thời gian lẫn bộ nhớ. Ngược lại, trên dữ liệu thưa như T10I4D100K, T20I6D100K và Retail, kích thước đầu ra tăng có kiểm soát, thách thức chính nằm ở việc duyệt và lọc qua nhiều giao dịch thay vì xử lý bùng nổ tổ hợp. Điều này phù hợp với anti-monotone property của support: khi $X \subseteq Y$ thì $\text{sup}(X) \geq \text{sup}(Y)$; trên dữ liệu dày, nhiều tập con đồng thời đạt ngưỡng nên số lượng frequent itemset tăng nhanh hơn.

4.5 Sử dụng bộ nhớ

Nhóm đo bộ nhớ đỉnh cho mỗi tập dữ liệu và so sánh giữa phiên bản Cơ bản và Tối ưu. Bảng 4.9 trình bày kết quả tại một ngưỡng MinSup đại diện cho mỗi tập dữ liệu.

Table 4.9: So sánh bộ nhớ đỉnh, MB giữa Cơ bản và Tối ưu

Dataset	MinSup	Basic (MB)	Optimized (MB)	Change
Connect-4	80%	279,93	239,28	−14,5%
Connect-4	95%	23,07	21,80	−5,5%
T10I4D100K	2%	19,76	2,36	−88,1%
T10I4D100K	5%	5,67	1,53	−73,0%
T20I6D100K	2%	146,50	113,34	−22,6%
Accidents	50%	161,92	161,65	$\approx 0\%$
Accidents	30%	215,66	268,57	+24,5%
Retail	30%	12,43	1,14	−90,8%
Retail	50%	18,40	2,24	−87,8%

Kết quả cho thấy trên dữ liệu thưa (T10I4D100K, Retail), Tối ưu giảm bộ nhớ rất hiệu quả nhờ cơ chế Stack-Buffered Lazy Allocation và BitArray Filtering, với mức giảm từ 73% đến 91%. Trên dữ liệu dày (Connect-4, Accidents), lợi ích bộ nhớ phụ thuộc vào ngưỡng MinSup: ở các ngưỡng trung bình và cao, Tối ưu tiết kiệm bộ nhớ; nhưng tại MinSup thấp khi kích thước đầu ra rất lớn, chi phí cấp phát cấu trúc trung gian của Tối ưu có thể vượt Cơ bản. Đây là đặc điểm đánh đổi cần lưu ý khi triển khai thuật toán trên dữ liệu dày với ngưỡng support thấp.

4.6 Khả năng mở rộng

Để đánh giá khả năng mở rộng, nhóm chọn tập dữ liệu Retail (88.162 giao dịch) và tạo các tập con với tỷ lệ 10%, 25%, 50%, 75% và 100% số giao dịch. Ngưỡng MinSup được cố định ở 50%. Thời gian thực thi của cả hai phiên bản (Julia Tối ưu và SPMF) được đo trên từng tập con.

Table 4.10: Khả năng mở rộng trên Retail (MinSup = 50%)

Subset	Trans.	Julia (s)	SPMF (s)
10%	8.817	0,002	0,145
25%	22.041	0,005	0,197
50%	44.081	0,011	0,290
75%	66.122	0,017	0,296
100%	88.162	0,021	0,385

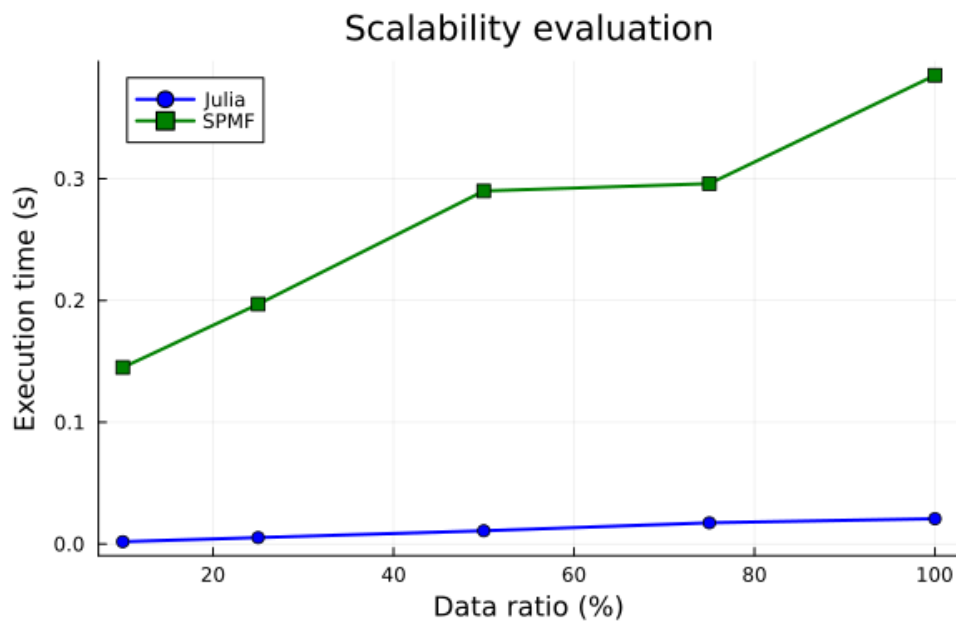


Figure 4.17: Đồ thị thời gian chạy theo kích thước dữ liệu trên Retail

Cả hai phiên bản đều thể hiện xu hướng thời gian chạy gần tuyến tính khi kích thước dữ liệu tăng. Khi số giao dịch tăng gấp 10 lần (từ 8.817 lên 88.162), thời gian của Julia tăng khoảng 10 lần (từ 0,002s lên 0,021s), xác nhận độ phức tạp thời gian thực tế đạt mức $O(N)$ trên dữ liệu thưa. Đường thời gian của Julia nằm gần trục hoành với hệ số hằng số nhỏ, trong khi SPMF có độ dốc cao hơn do chi phí khởi động JVM và cấp phát đối tượng Java. Xu hướng tuyến tính này phù hợp với lý thuyết: trên dữ liệu

thừa, FP-tree có ít nhánh chung sâu, kích thước cây tăng tuyến tính theo số giao dịch đầu vào, nên chi phí duyệt cây và khai thác đệ quy duy trì ở mức tuyến tính.

4.7 Ảnh hưởng của độ dài giao dịch

Nhóm thiết kế thực nghiệm trên dữ liệu tổng hợp với cấu hình cố định: 100.000 giao dịch, 1.000 items, MinSup = 10%, và độ dài giao dịch trung bình tăng dần từ 10 đến 40.

Table 4.11: Ảnh hưởng của độ dài giao dịch trung bình đến hiệu năng

Avg. length	Julia (s)	Julia (MB)	SPMF (s)	SPMF (MB)
10	0,015	19,45	0,288	29,06
20	0,028	3,44	2,218	99,30
30	0,045	3,70	0,574	42,05
40	0,059	14,58	0,666	110,16

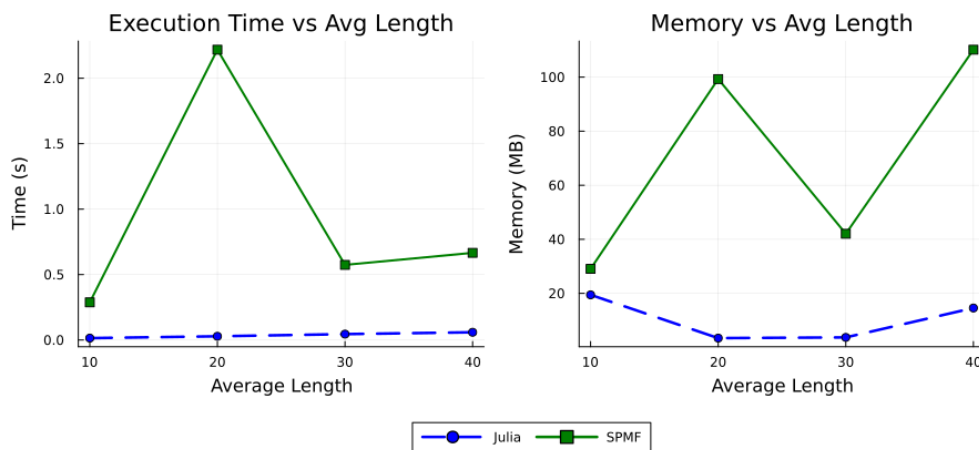


Figure 4.18: Ảnh hưởng của độ dài giao dịch trung bình đến thời gian và bộ nhớ

Khi độ dài giao dịch trung bình tăng từ 10 lên 40, thời gian xử lý của Julia tăng nhẹ từ 0,015s lên 0,059s, trong khi SPMF biến động mạnh hơn. Về bộ nhớ, Julia duy trì mức tiêu thụ gần như ổn định ở mức thấp, trong khi SPMF dao động lớn (từ 29,06MB đến 110,16MB). Kết quả này phù hợp với lý thuyết: khi giao dịch dài hơn, số nút trong FP-tree tăng, các đường đi chung trở nên kém nén hơn, tăng chi phí chèn giao dịch và khai thác cây điều kiện. Tuy nhiên, cài đặt Julia sử dụng BitArray có kích thước cố định theo không gian item (1.000 items) nên bộ nhớ ít biến động; kết hợp với Buffer Reuse giúp hạn chế phát sinh cấp phát mới khi độ dài giao dịch tăng.

4.8 Phân tích tổng hợp kết quả

4.8.1 Giải thích kết quả dựa trên lý thuyết

Thuật toán FP-Growth xây dựng FP-tree từ các items phổ biến và khai thác thông qua conditional pattern-base cùng conditional FP-tree để liệt kê đầy đủ frequent itemset mà không sinh ứng viên. Khi ngưỡng ξ giảm, anti-monotone property của support ($X \subseteq Y \Rightarrow \text{sup}(X) \geq \text{sup}(Y)$) khiến nhiều tập con trước đó không đủ điều kiện trở thành phổ biến, dẫn đến bùng nổ tổ hợp. Trên dữ liệu dày, mỗi giao dịch chứa nhiều items nên FP-tree có nhiều nhánh và đường đi chung dài, làm tăng chi phí xây dựng cây điều kiện. Ngược lại, trên dữ liệu thưa, FP-tree gọn nhẹ hơn do ít nhánh chung sâu, chi phí khai thác tăng chậm theo kích thước dữ liệu.

Các kỹ thuật tối ưu hóa phát huy hiệu quả ở những ngữ cảnh khác nhau. Single Path Pruning hiệu quả nhất khi conditional FP-tree thu gọn thành đường đơn, cho phép liệt kê toàn bộ tập lũy thừa bằng phép toán bit thay vì đệ quy sâu. BitArray Filtering giảm chi phí kiểm tra tư cách phổ biến từ tra cứu bảng băm sang truy cập mảng trực tiếp, đặc biệt hiệu quả khi duyệt qua nhiều giao dịch. Stack-Buffered Lazy Allocation triệt tiêu chi phí cấp phát động tại mỗi tầng đệ quy, mang lại lợi ích rõ nhất khi cây đệ quy sâu.

4.8.2 Điểm mạnh và điểm yếu so với SPMF

Điểm mạnh của cài đặt Julia bao gồm tốc độ xử lý nhanh hơn SPMF ở phần lớn các cấu hình thử nghiệm, đặc biệt trên dữ liệu thưa; mức tiêu thụ bộ nhớ thấp hơn đáng kể trên dữ liệu thưa nhờ cấu trúc dữ liệu gọn và cơ chế tái sử dụng bộ đệm; xu hướng mở rộng tuyến tính ổn định khi kích thước dữ liệu tăng; và tính đúng đắn tuyệt đối được kiểm chứng qua đối chiếu 100% với SPMF.

Điểm yếu cần lưu ý là trên dữ liệu dày với MinSup thấp, chi phí bộ nhớ của Tối ưu có thể vượt cả Cơ bản lẫn SPMF do cấp phát cấu trúc trung gian. Cụ thể, trên Connect-4 tại MinSup = 75%, bộ nhớ của Julia đạt khoảng 835MB trong khi SPMF chỉ dùng khoảng 31MB. Ngoài ra, cài đặt hiện tại chưa tận dụng song song hóa ở bước khai thác các conditional FP-tree độc lập, và hệ sinh thái chưa trưởng thành bằng SPMF về số lượng thuật toán và tùy chọn hậu xử lý.

4.8.3 Hướng tối ưu hóa tiếp theo

Dựa trên phân tích kết quả, nhóm đề xuất các hướng tối ưu hóa cụ thể sau. Thứ nhất, song song hóa khai thác conditional FP-tree theo nhánh items hậu tố bằng cơ chế đa luồng, kèm gộp kết quả cuối để giảm thời gian ở ngưỡng MinSup thấp trên dữ liệu dày. Thứ hai, hỗ trợ chế độ xuất kết quả theo dòng (ghi kết quả tuần tự) thay vì giữ toàn bộ frequent itemset trong RAM, giúp giảm bộ nhớ đỉnh khi kích thước đầu ra rất lớn. Thứ ba, bổ sung tùy chọn khai thác tập đóng hoặc tập tối đại để giảm kích thước đầu ra và giảm chi phí hậu xử lý. Thứ tư, gộp các giao dịch trùng lặp trước khi mining và lưu thêm trọng số xuất hiện để giảm số giao dịch phải chèn vào FP-tree khi dữ liệu có nhiều mẫu lặp.

5. Ứng dụng — Market Basket Analysis

Chương này trình bày việc áp dụng thuật toán FP-Growth do nhóm cài đặt vào bài toán Market Basket Analysis trên dữ liệu bán lẻ thực tế, nhằm khai thác các frequent itemset, sinh association rule và rút ra ý nghĩa kinh doanh từ các mẫu hình mua sắm phổ biến.

5.1 Giới thiệu bài toán

Market Basket Analysis là kỹ thuật khai phá dữ liệu nhằm phát hiện các mối quan hệ ẩn giữa các sản phẩm được mua cùng lúc trong một giao dịch. Item tiêu chính là tìm ra các nhóm sản phẩm thường xuyên đồng xuất hiện, từ đó sinh các association rule dạng $X \Rightarrow Y$ có ý nghĩa thực tiễn cho hoạt động kinh doanh. Các ứng dụng điển hình bao gồm tối ưu hóa bố trí kệ hàng, thiết kế chương trình khuyến mãi chéo, gợi ý sản phẩm cá nhân hóa và quản trị tồn kho dựa trên mẫu hình tiêu dùng.

Trong chương này, nhóm sử dụng thuật toán FP-Growth tối ưu, được cài đặt bằng Julia, để khai thác frequent itemset từ dữ liệu giao dịch siêu thị, sau đó sinh association rule và phân tích mười rule có giá trị lift cao nhất. Mỗi rule được giải thích về ý nghĩa kinh doanh thực tiễn, bao gồm tâm lý tiêu dùng, gợi ý trưng bày và chiến lược khuyến mãi.

5.2 Tập dữ liệu và tiền xử lý

5.2.1 Mô tả tập dữ liệu

Nhóm sử dụng tập dữ liệu Bộ dữ liệu Groceries từ Kaggle, chứa dữ liệu mua hàng thực tế tại một chuỗi siêu thị tạp hóa. Bảng 5.1 trình bày thông tin tổng quan.

Table 5.1: Thông tin tập dữ liệu Groceries

Thông tin	Chi tiết
Nguồn	Bộ dữ liệu Groceries từ Kaggle
Số hàng gốc	38.765 hàng vi mua hàng
Số khách hàng	3.898
Số sản phẩm	167 loại
Đơn vị giao dịch	Một lượt mua theo (Member_number, Date)

5.2.2 Chiến lược gom giao dịch

Mỗi giao dịch được định nghĩa là một lượt mua theo cặp (Member_number, Date), tức tập hợp các sản phẩm mà một khách hàng mua trong cùng một ngày. Cách gom này phản ánh hành vi mua đồng thời, phù hợp với bài toán Market Basket Analysis. Sau khi gom theo chiến lược trên, tập dữ liệu chuyển thành 14.963 giao dịch.

5.2.3 Mã hóa item

Toàn bộ 167 sản phẩm được sắp xếp theo thứ tự bảng chữ cái và mã hóa thành các số nguyên từ 1 đến 167 theo định dạng đầu vào chuẩn của thuật toán FP-Growth.

5.2.4 Phân phối kích thước giỏ hàng

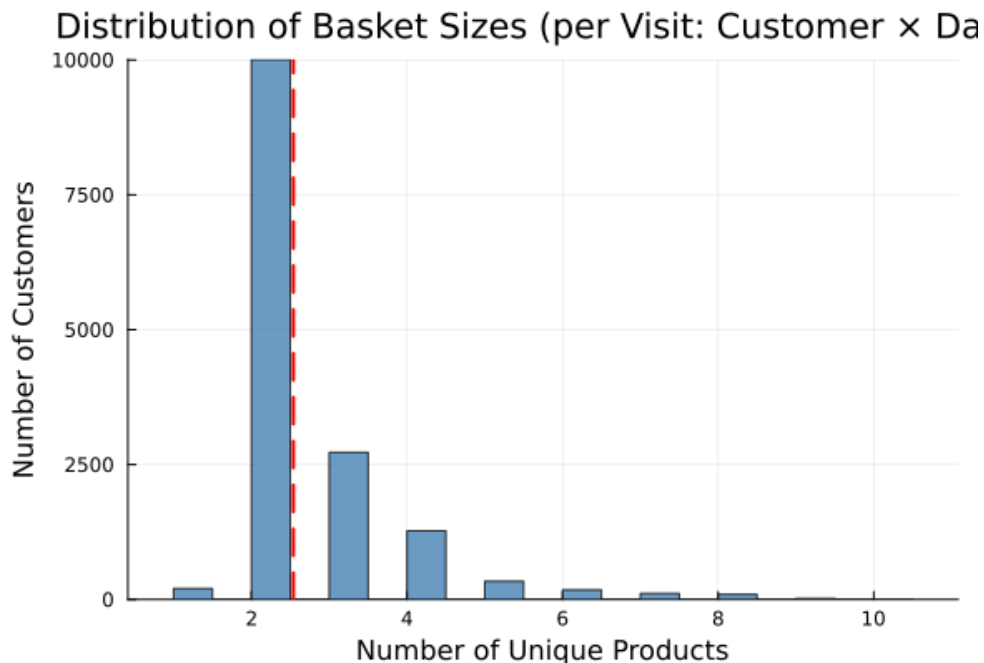


Figure 5.1: Phân phối kích thước giỏ hàng (số sản phẩm mỗi lượt mua)

Hình 5.1 cho thấy phân phối lệch phải rõ rệt: phần lớn giao dịch chỉ chứa từ 1 đến 3 sản phẩm, với kích thước trung bình là 2,54 và tối đa là 10. Đa số khách hàng ghé siêu thị để mua một vài món vặt vặt thay vì mua số lượng lớn cùng lúc. Consequent đối với khai phá frequent itemset: tính phân tán và thừa thớt đòi hỏi ngưỡng MinSup đủ thấp để có thể phát hiện các mẫu hình đồng xuất hiện có ý nghĩa.

5.2.5 Phân tích thống kê theo thời gian

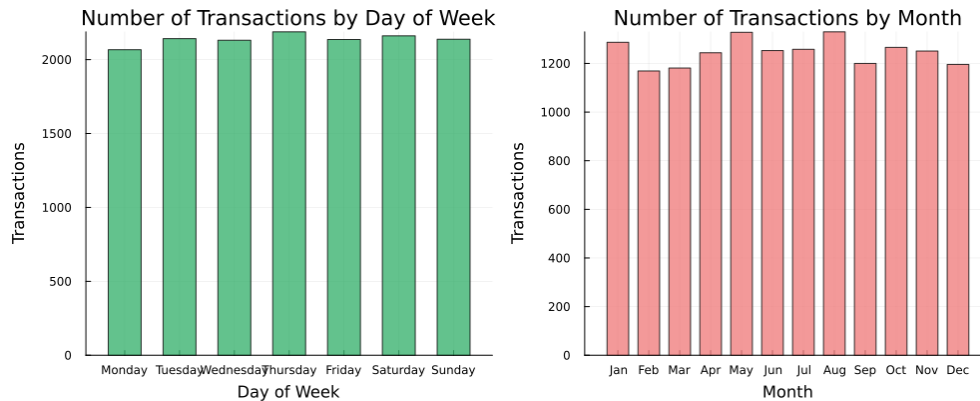


Figure 5.2: Phân bố giao dịch theo thứ trong tuần và theo tháng

Hình 5.2 trình bày phân bố giao dịch theo hai chiều thời gian. Mật độ giao dịch theo thứ trong tuần thể hiện tính phân phối đều, không có sự gia tăng đột biến vào cuối tuần. Phân bố theo tháng cũng duy trì tính ổn định cao, không xuất hiện chu kỳ mùa vụ rõ ràng. Đặc trưng này phản ánh mô hình bán lẻ hướng đến nhu cầu vi mô thường nhật: khách hàng duy trì tần suất ghé thăm ổn định để bổ sung hàng tiêu dùng nhanh thay vì gom mua sắm quy mô lớn vào dịp cuối tuần hay mùa lễ. Do đó, việc xây dựng association rule nên tập trung vào sự bổ trợ công năng giữa các sản phẩm thay vì chiến lược khuyến mãi phụ thuộc chu kỳ thời gian.

5.3 Lựa chọn ngưỡng tham số

Nhóm lựa chọn ngưỡng $MinSup = 0,001$ (0,1%) và $confidencethreshold = 0,05$ (5%) dựa trên các đặc trưng dữ liệu sau.

Về $MinSup = 0,1\%$: Groceries tập dữ liệu rất thưa với mỗi lượt mua chỉ trung bình 2,54 sản phẩm so với 167 loại. Nếu đặt ngưỡng quá cao (1% hay 5%), nhiều cặp sản phẩm có ý nghĩa khám phá sẽ bị loại bỏ. Ngưỡng 0,1% tương đương ít nhất 15 giao dịch hỗ trợ, phù hợp cho phân tích khám phá ở mức vi mô.

Về $confidence threshold = 5\%$: Ngưỡng confidence 5% giúp giữ lại đủ rule để quan sát cấu trúc dữ liệu trong bối cảnh basket rất nhỏ. Tuy nhiên đây là ngưỡng thấp, các rule sinh ra cần được lọc tiếp bằng lift, absolute support và kiểm tra độ ổn định trước khi dùng cho quyết định kinh doanh.

5.4 Khai thác frequent itemset

Nhóm chạy FP-Growth tối ưu trên 14.963 giao dịch với MinSup tuyệt đối ≥ 15 giao dịch. Kết quả thu được 750 frequent itemset trong thời gian 1,085s. Bảng 5.2 trình bày phân bố theo kích thước.

Table 5.2: Phân bố frequent itemset theo kích thước

Kích thước	Số lượng
1-itemset	149
2-itemset	592
3-itemset	9
Tổng	750

Phần lớn frequent itemset có kích thước 2 (chiếm 78,9%), phản ánh đặc trưng giỏ hàng nhỏ của dữ liệu. Chỉ có 9 frequent itemset kích thước 3, cho thấy rất ít tổ hợp 3 sản phẩm cùng đạt ngưỡng support trong bối cảnh dữ liệu thưa.

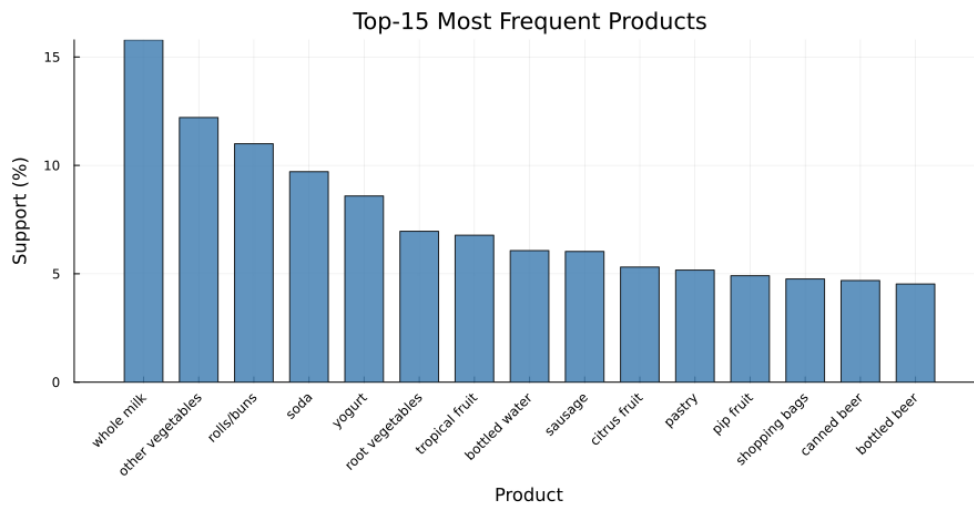


Figure 5.3: Top-15 sản phẩm phổ biến nhất theo support

Hình 5.3 cho thấy các sản phẩm nhu yếu phẩm như sữa tươi, xúc xích, sữa chua, rolls/buns và nước ngọt có ga chiếm vị trí đầu bảng, phù hợp với đặc trưng siêu thị tạp hóa phục vụ nhu cầu hàng ngày.

5.5 Sinh association rule

Từ 750 frequent itemset, nhóm sinh association rule $X \Rightarrow Y$ và đánh giá bằng ba chỉ số. Support được tính bằng $\text{sup}(X \cup Y)/N$, thể hiện tỷ lệ giao dịch chứa cả X và

Y . Confidence được tính bằng $\text{sup}(X \cup Y)/\text{sup}(X)$, thể hiện xác suất mua Y khi đã mua X . Lift được tính bằng $\text{conf}(X \Rightarrow Y)/\text{sup}(Y)$, trong đó $\text{lift} > 1$ biểu thị tương quan dương thực sự giữa X và Y .

Quá trình sinh rule tạo ra tổng cộng 450 association rule thỏa ngưỡng confidence threshold = 5%. Trong đó, 94 rule (20,89%) có lift > 1 và giá trị lift trung bình của toàn bộ tập rule là 0,8859. Điều này phản ánh thực tế khách quan: phần lớn hành vi mua sắm tại siêu thị mang tính ngẫu nhiên hoặc độc lập đối với đại đa số sản phẩm. Nhóm nhỏ các rule có lift > 1 chính là những mẫu hình thể hiện mối tương quan dương thực sự.

5.6 10 rule theo lift

Bảng 5.3 trình bày 10 association rule có giá trị lift cao nhất.

Table 5.3: 10 association rule theo lift

#	Antecedent	$\Rightarrow$	Consequent	Count	Sup%	Conf%	Lift
1	sữa tươi + sữa chua	$\Rightarrow$	xúc xích	22	0,147	13,17	2,183
2	xúc xích + sữa tươi	$\Rightarrow$	sữa chua	22	0,147	16,42	1,912
3	specialty chocolate	$\Rightarrow$	citrus fruit	21	0,140	8,79	1,654
4	xúc xích + sữa chua	$\Rightarrow$	sữa tươi	22	0,147	25,58	1,620
5	bột mì	$\Rightarrow$	trái cây nhiệt đới	16	0,107	10,96	1,617
6	đồ uống	$\Rightarrow$	xúc xích	23	0,154	9,27	1,537
7	nước ngọt có ga + sữa tươi	$\Rightarrow$	xúc xích	16	0,107	9,20	1,524
8	khăn giấy	$\Rightarrow$	bánh ngọt	26	0,174	7,85	1,519
9	phô mai chế biến	$\Rightarrow$	rau củ dạng rễ	16	0,107	10,53	1,513
10	phô mai cứng	$\Rightarrow$	trái cây có hạt	16	0,107	7,27	1,483

5.6.1 Phân tích ý nghĩa kinh doanh

Rule 1: sữa tươi + sữa chua $\Rightarrow$ xúc xích (lift = 2,183). Khi khách hàng mua đồng thời sữa tươi và sữa chua, xác suất họ mua thêm xúc xích tăng gấp 2,18 lần so với mức nền. Sữa tươi và sữa chua đại diện cho ngành hàng bơ sữa, thường được hộ gia đình có trẻ nhỏ mua để chuẩn bị bữa sáng. Xúc xích đóng vai trò nguồn protein tiện lợi, hoàn thiện bức tranh “Bữa sáng dinh dưỡng nhanh gọn cho gia đình”. Gợi ý kinh doanh: thiết lập gói combo “Bữa Sáng Tiện Lợi” gồm sữa tươi, sữa chua và xúc xích; đặt kệ xúc xích ăn liền ngay cạnh quầy sữa mát.

Rule 2: xúc xích + sữa tươi $\Rightarrow$ sữa chua (lift = 1,912). Nếu giỏ hàng đã chứa xúc xích và sữa tươi, xác suất mua thêm sữa chua đạt 16,42%, gấp 1,91 lần mức

bình quân. Rule này củng cố giả thuyết về nhóm sản phẩm bữa sáng gia đình. Gợi ý: đặt lốc sữa chua nhỏ tại tủ mát cạnh quầy thanh toán; khi hệ thống POS phát hiện hóa đơn có sữa tươi và xúc xích, nhân viên gợi ý thêm sữa chua.

Rule 3: specialty chocolate $\Rightarrow$ citrus fruit (lift = 1,654). Khách hàng mua sô-cô-la đặc biệt cao cấp có xu hướng mua kèm trái cây họ cam quýt, gấp 1,65 lần mức thông thường. Hương vị đậm đà của sô-cô-la đen hòa quyện với vị chua thanh của cam là sự kết hợp ẩm thực kinh điển. Phân khúc khách hàng này thường có lối sống tinh tế. Gợi ý: trưng bày sô-cô-la cao cấp tại khu vực cam quýt nhập khẩu; thiết kế giỏ quà tặng Gourmet kết hợp hai sản phẩm.

Rule 4: xúc xích + sữa chua $\Rightarrow$ sữa tươi (lift = 1,620). Khi giỏ hàng có xúc xích và sữa chua, 25,58% khách hàng mua thêm sữa tươi. Sữa tươi là sản phẩm đích thu hút khách hàng, thường đặt sâu trong siêu thị. Gợi ý: bố trí quầy xúc xích và sữa chua trên luồng đi dẫn đến quầy sữa tươi để tối đa hóa khả năng khách nhặt cả ba mặt hàng.

Rule 5: bột mì $\Rightarrow$ trái cây nhiệt đới (lift = 1,617). Người mua bột mì có xác suất mua kèm trái cây nhiệt đới gấp 1,62 lần mức bình thường, phản ánh mục đích làm bánh tại gia. Bột mì là nguyên liệu cốt lõi; chuối, xoài, dứa là nguyên liệu nhân bánh phổ biến. Gợi ý: thiết kế tờ công thức làm bánh đặt tại quầy bột mì; trưng bày hoa quả nhiệt đới cắt sẵn ngay cạnh khu nguyên liệu làm bánh.

Rule 6: đồ uống $\Rightarrow$ xúc xích (lift = 1,537). Khách mua đồ uống đóng chai có xu hướng mua kèm xúc xích gấp 1,54 lần, đại diện cho phân khúc tiêu dùng nhanh và tiện lợi. Gợi ý: đặt móc treo xúc xích đóng gói nhỏ cạnh tủ mát nước ngọt; thiết lập combo đồng giá “1 chai nước + 1 cây xúc xích” hướng đến học sinh, sinh viên.

Rule 7: nước ngọt có ga + sữa tươi $\Rightarrow$ xúc xích (lift = 1,524). Sự xuất hiện đồng thời của nước ngọt có ga và sữa tươi biểu thị giỏ hàng gia đình hoặc chuẩn bị họp mặt cuối tuần. Xúc xích là món ăn nhẹ phổ biến, dễ chế biến. Gợi ý: đề xuất gói xúc xích túi lớn khi hệ thống phát hiện khách đặt mua lốc nước ngọt và sữa tươi dung tích lớn.

Rule 8: khăn giấy $\Rightarrow$ bánh ngọt (lift = 1,519). Người mua khăn giấy có xác suất mua kèm bánh ngọt gấp 1,52 lần, phản ánh ngữ cảnh sử dụng trực tiếp: bánh ngọt nhiều bơ, dễ dính tay cần khăn giấy khi thưởng thức. Đây là rule có count cao nhất trong 10 (26 giao dịch). Gợi ý: đặt giá khăn giấy bỏ túi ngay cạnh quầy bánh ngọt tự chọn; tặng kèm khăn giấy khi đóng gói bánh mang đi.

Rule 9: phô mai chế biến $\Rightarrow$ rau củ dạng rẽ (lift = 1,513). Khách mua phô mai chế biến có xu hướng mua kèm rau củ dạng rẽ gấp 1,51 lần, biểu trưng cho

nhu cầu nấu ăn gia đình. Phô mai lát thường dùng làm nguyên liệu phủ khoai tây dứt lò hoặc súp rau củ. Gợi ý: đặt tủ mát nhỏ trưng bày phô mai chế biến ngay tại khu vực rau củ tươi; thiết kế biển gợi ý công thức nấu ăn.

Rule 10: phô mai cứng $\Rightarrow$ trái cây có hạt (lift = 1,483). Khách mua phô mai cứng có xu hướng mua kèm táo hoặc lê gấp 1,48 lần, phản ánh sự kết hợp ẩm thực kinh điển phương Tây: đĩa phô mai với trái cây giòn. Gợi ý: thiết lập ụ trưng bày “Đĩa tiệc Cheese & Wine” kết hợp phô mai cứng cao cấp, táo tươi và rượu vang; đóng gói khay đồ ăn nhẹ phô mai và trái cây cho dân văn phòng.

5.7 Trực quan hóa

Nhóm thiết kế năm biểu đồ nhằm cung cấp góc nhìn trực quan đa chiều về tập association rule.

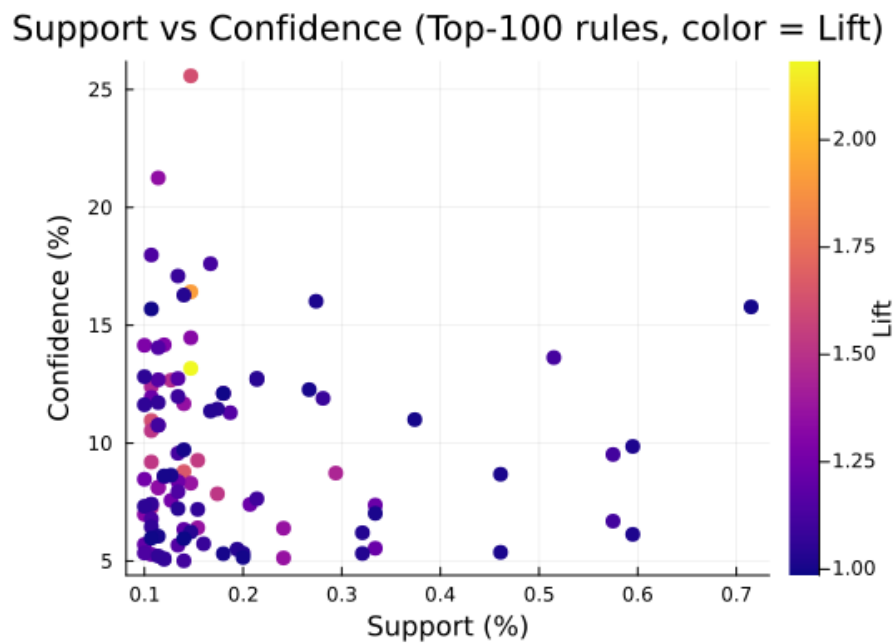


Figure 5.4: Biểu đồ phân tán giữa support và confidence của 100 luật; màu biểu thị lift

Hình 5.4 trình bày biểu đồ phân tán giữa support và confidence của 100 rule hàng đầu, với màu sắc mã hóa giá trị lift. Các rule có lift cao tập trung ở vùng support thấp và confidence trung bình, phản ánh đặc trưng dữ liệu thưa: các mẫu hình mạnh nhất thường xuất hiện ở tần suất thấp nhưng có mức tương quan dương cao.

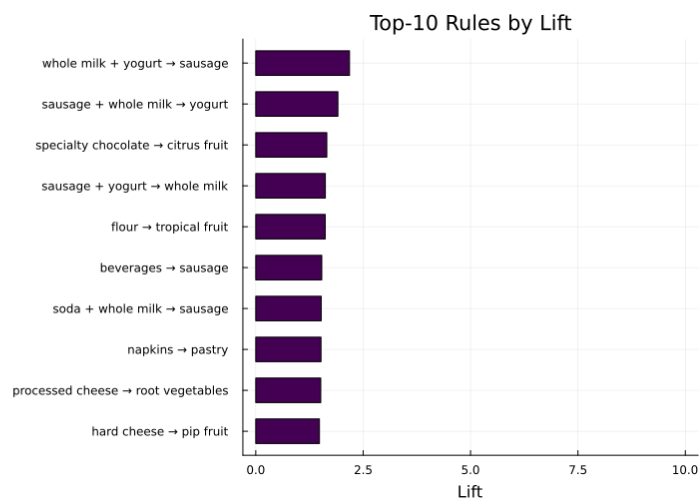


Figure 5.5: 10 rule hàng đầu theo lift bằng biểu đồ thanh ngang

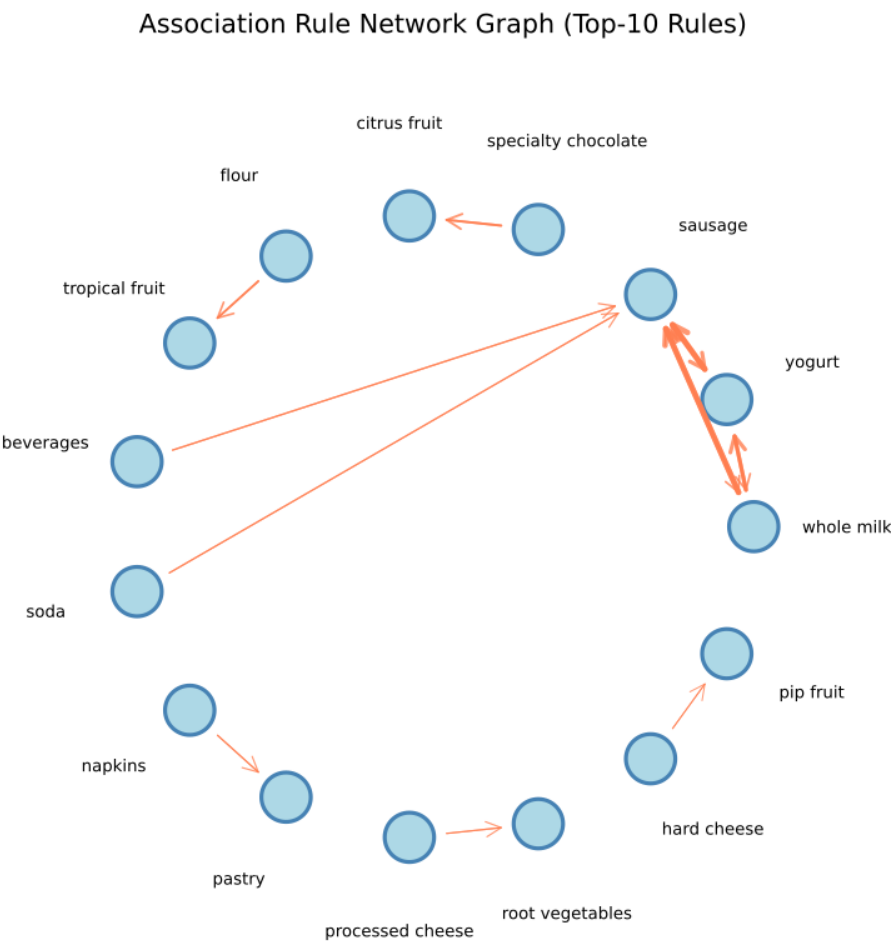


Figure 5.6: Đồ thị mạng association rule của mười rule hàng đầu

Hình 5.6 biểu diễn mối quan hệ giữa các sản phẩm dưới dạng đồ thị mạng. Mỗi nút là một sản phẩm, mỗi cạnh có hướng thể hiện một association rule với độ dày tỷ lệ thuận với lift. Đồ thị cho thấy sản phẩm sữa tươi, xúc xích và sữa chua đóng vai trò

trung tâm với nhiều kết nối, phù hợp với vai trò nhu yếu phẩm thiết yếu của chúng.

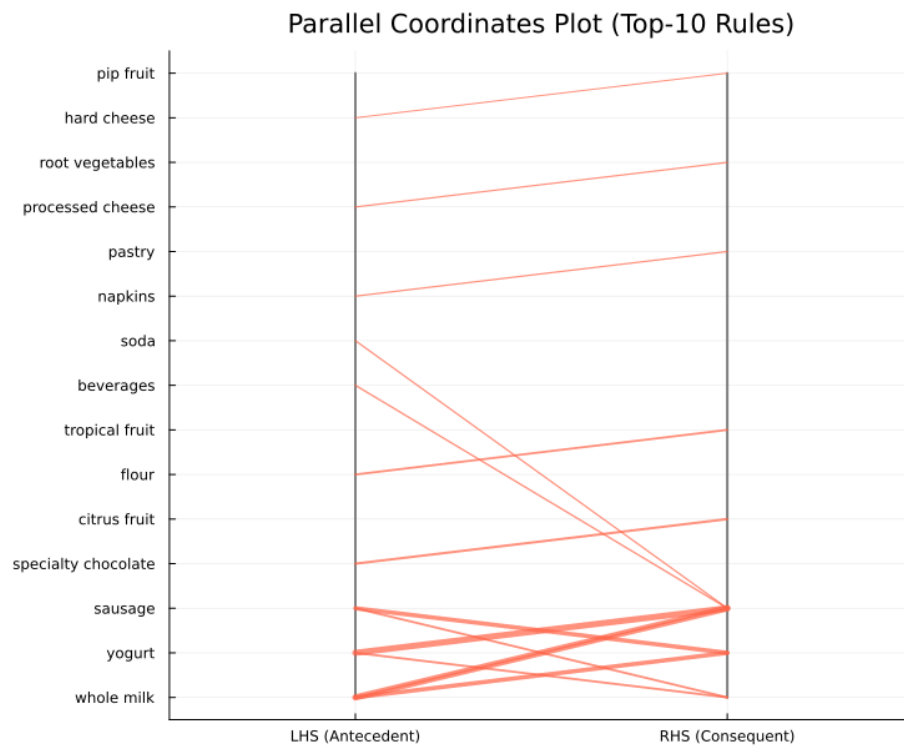


Figure 5.7: Biểu đồ tọa độ song song của mười rule hàng đầu

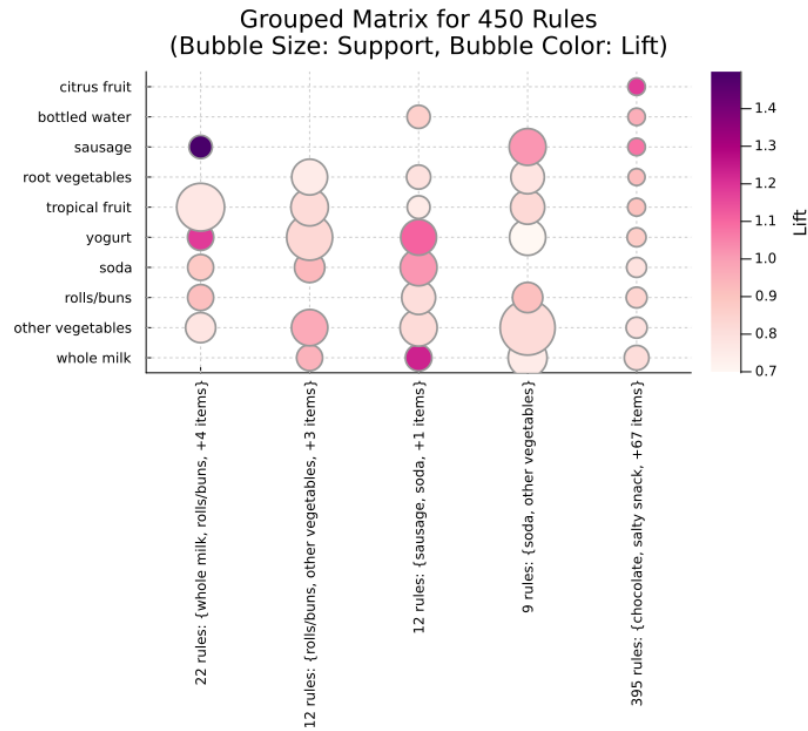


Figure 5.8: Ma trận nhóm cho toàn bộ 450 rule với kích thước biểu thị support và màu biểu thị lift

Hình 5.8 trình bày biểu đồ ma trận nhóm, trong đó trục hoành chia các rule theo nhóm antecedent phổ biến nhất và trục tung là các consequent phổ biến. Kích thước bong bóng thể hiện support trung bình và màu sắc thể hiện lift. Biểu đồ cho thấy nhóm rule liên quan đến sữa tươi và xúc xích chiếm tỷ trọng lớn nhất, với các consequent chính là sữa chua, xúc xích và sữa tươi.

5.8 Hạn chế và hướng mở rộng

5.8.1 Hạn chế của mô hình

Mô hình hiện tại có một số hạn chế cần lưu ý. Thứ nhất, các rule trong nhóm mười rule hàng đầu có số lượng giao dịch hỗ trợ nhỏ, từ 16 đến 26 giao dịch, nên rất nhạy cảm với nhiễu dữ liệu. Một số rule có thể sinh ra do trùng hợp ngẫu nhiên tại một số cửa hàng trong khoảng thời gian nhất định chứ không phải xu hướng bền vững.

Thứ hai, mô hình chạy cố định ở một cấu hình ngưỡng duy nhất, với MinSup 0,1% và confidence threshold 5%, mà chưa có phân tích nhạy cảm (phân tích nhạy cảm) để xác định cấu hình tối ưu một cách khoa học.

Thứ ba, lift trung bình chỉ đạt 0,8859 cho thấy phần lớn rule gần 80% không thể hiện tương quan dương, phản ánh giới hạn nội tại của bài toán trên dữ liệu rất thưa với giỏ hàng nhỏ.

5.8.2 Hướng mở rộng

Để nâng cao confidence và giá trị ứng dụng, nhóm đề xuất các hướng cải tiến sau. Về kiểm tra độ ổn định: phân tách dữ liệu theo thời gian thành tập huấn luyện và tập kiểm tra để đánh giá xem các lift và confidence của 10 rule có duy trì ổn định hay không; kết hợp phương pháp Bootstrap để tính khoảng tin cậy cho từng chỉ số.

Về phân tích nhạy cảm: quét qua nhiều mức dải tham số với MinSup từ 0,05% đến 0,5% và confidence threshold từ 2% đến 20% và vẽ biểu đồ đánh đổi giữa số lượng rule sinh ra và chất lượng trung bình.

Về triển khai kinh doanh: áp dụng thử nghiệm A/B tại 3–5 cửa hàng đại diện trong 2–4 tuần trước khi nhân rộng toàn chuỗi; sử dụng hệ thống CRM để gửi phiếu giảm giá cá nhân hóa thay vì áp dụng giảm giá đại trà, vì support của các rule còn thấp.

Về mở rộng kỹ thuật: kết hợp phân tích chuỗi thời gian để phát hiện mẫu hình mua sắm theo thời điểm; bổ sung phân tích theo phân khúc khách hàng để tìm rule đặc thù cho từng nhóm; mở rộng sang khai thác mẫu tuần tự để phát hiện trình tự mua hàng.

Tài liệu tham khảo

Bibliography

- [1] J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation,” *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pp. 1–12, 2000.
- [2] J. Han, J. Pei, Y. Yin, and R. Mao, “Mining frequent patterns without candidate generation: A frequent-pattern tree approach,” *Data Mining and Knowledge Discovery*, vol. 8, no. 1, pp. 53–87, 2004.
- [3] R. Agrawal, T. Imieliński, and A. Swami, “Mining association rules between sets of items in large databases,” *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pp. 207–216, 1993.
- [4] P. Fournier-Viger et al., “The SPMF open-source data mining library,” *Journal of Machine Learning Research*, vol. 15, pp. 3389–3393, 2014. Trang dữ liệu và công cụ: <https://www.philippe-fournier-viger.com/spmf/>.
- [5] D. Dua and C. Graff, “UCI Machine Learning Repository,” University of California, Irvine, School of Information and Computer Sciences, 2019. <https://archive.ics.uci.edu/>.
- [6] Kaggle, “Groceries Dataset,” bộ dữ liệu dùng cho Market Basket Analysis. <https://www.kaggle.com/>.